

第三部分：HTTP/2 协议



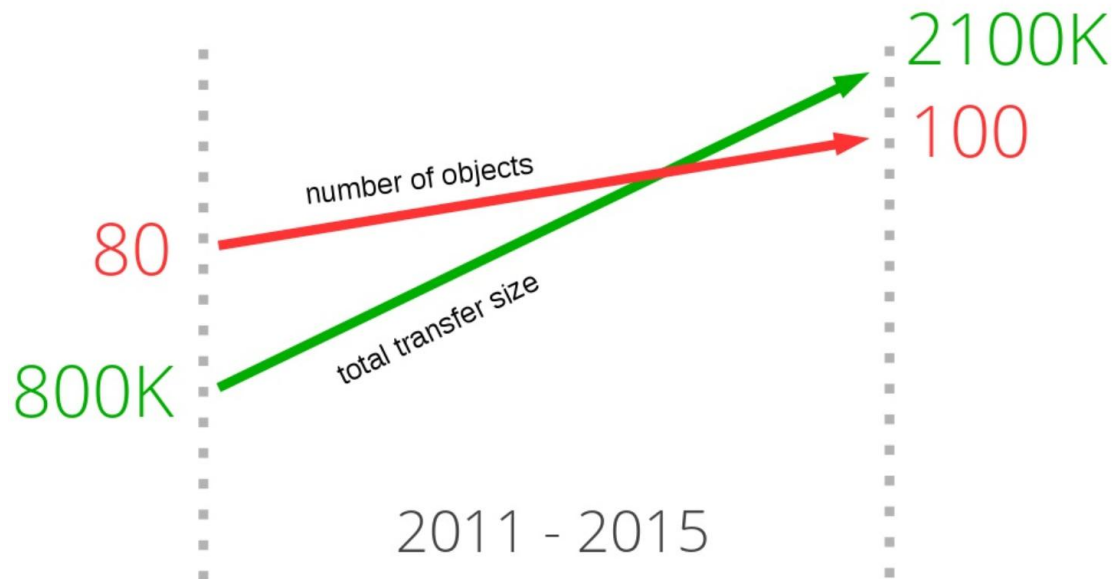
扫码试看/订阅极客时间

《Web协议详解与抓包实战》视频课程

第 1 课 HTTP/1.1 发展中遇到的问题

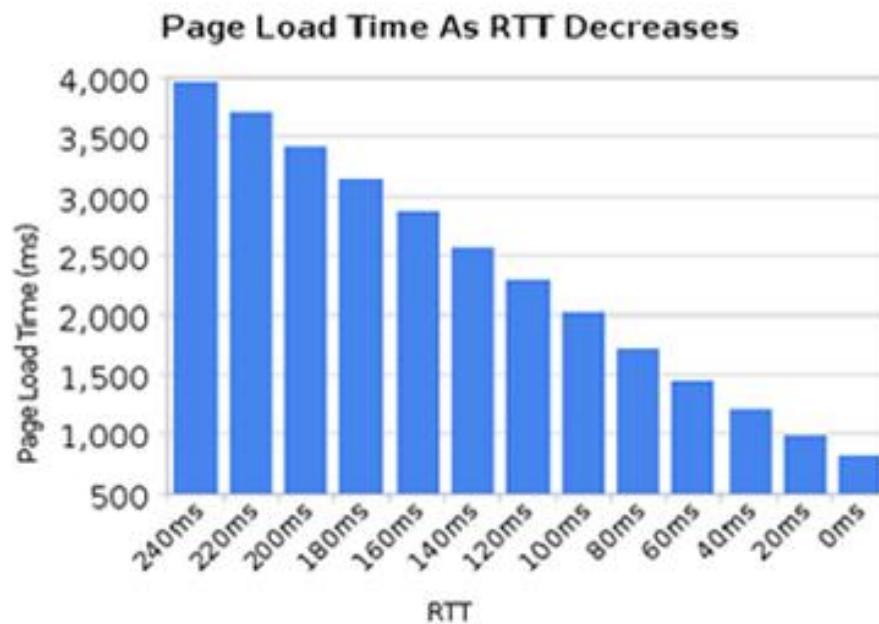
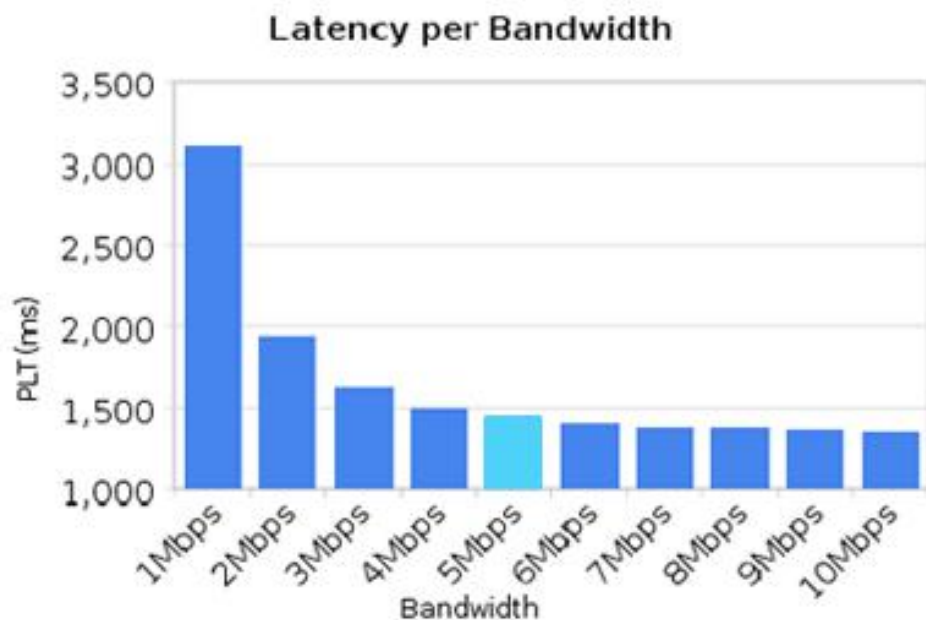
HTTP/1.1 发明以来发生了哪些变化？

- 从几 KB 大小的消息，到几 MB 大小的消息
- 每个页面小于 10 个资源，到每页面 100 多个资源
- 从文本为主的内容，到富媒体（如图片、声音、视频）为主的内容
- 对页面内容实时性高要求的应用越来越多



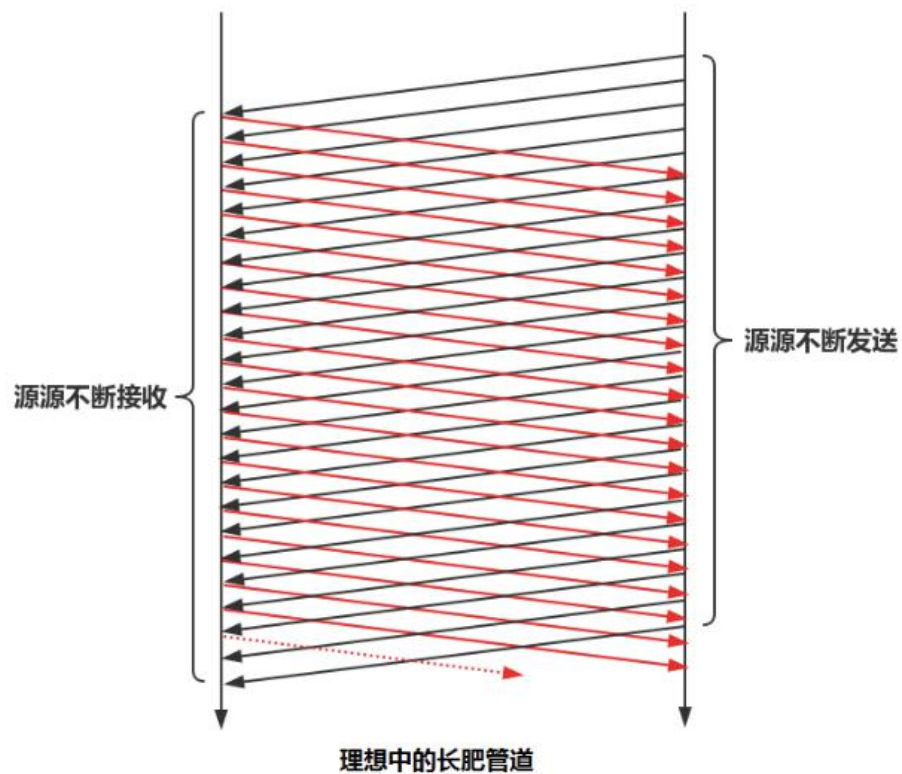
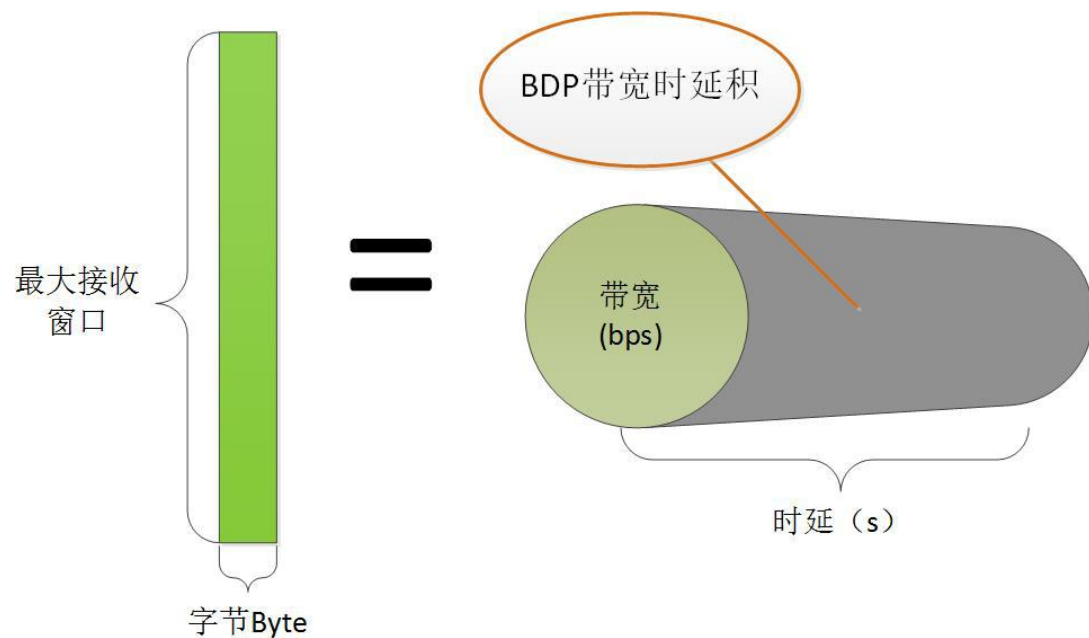
HTTP/1.1 的高延迟问题

- 高延迟带来页面加载速度的降低
 - 随着带宽的增加，延迟并没有显著下降
 - 并发连接有限
 - 同一连接同时只能在完成一个 HTTP 事务（请求/响应）才能处理下一个事务



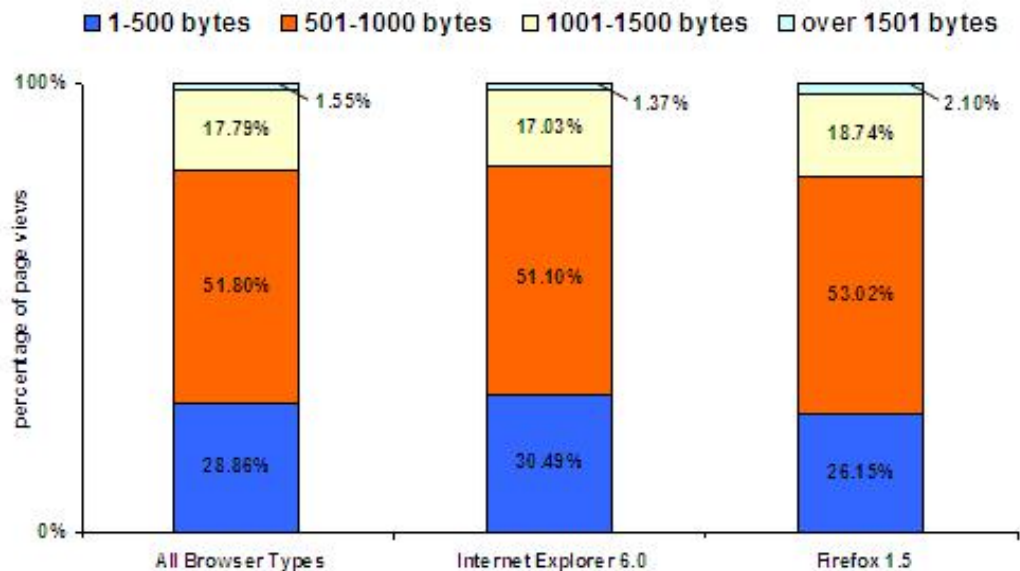
高延迟 VS 高带宽

- 单连接上的串行请求
- 无状态导致的高传输量（低网络效率）



无状态特性带来的巨大 HTTP 头部

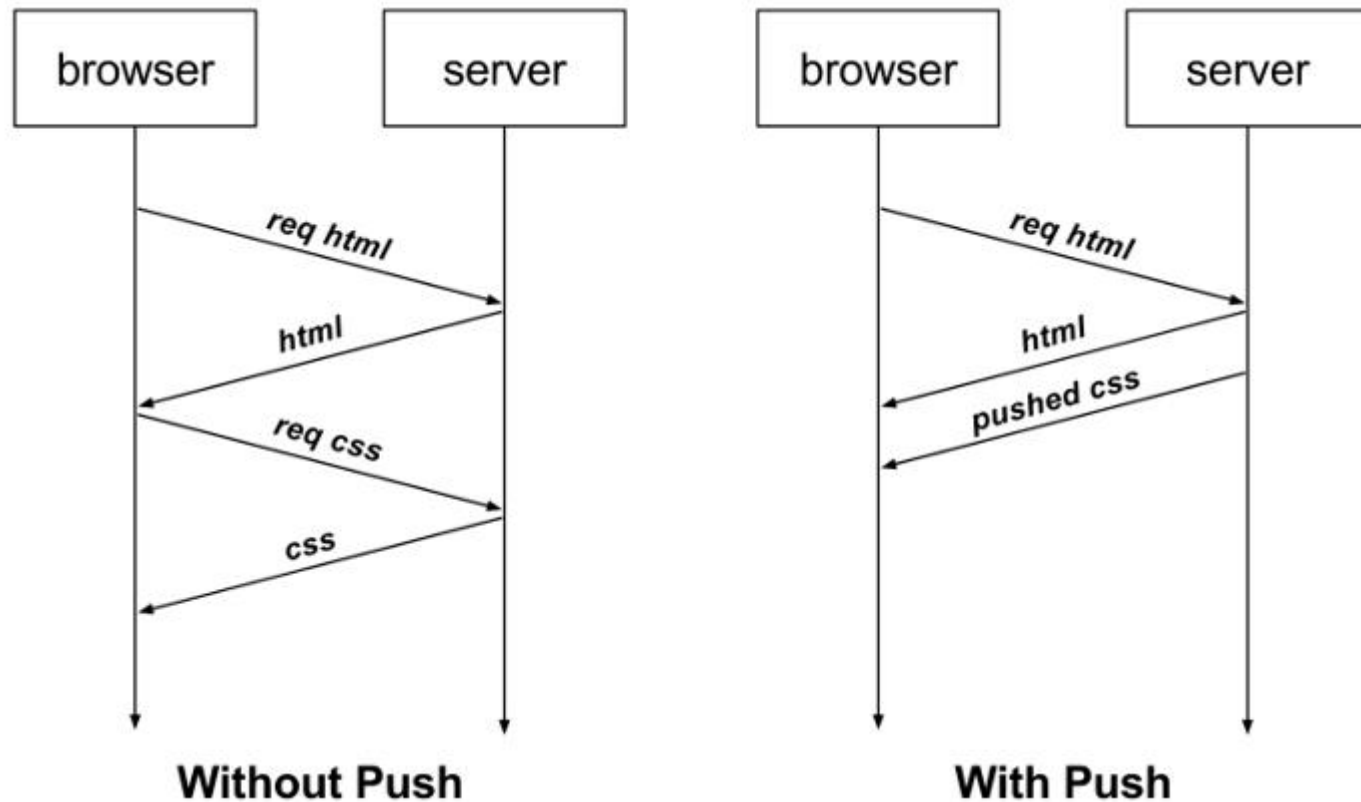
- 重复传输的体积巨大的 HTTP 头部



HTTP/1.1 为了解决性能问题做过的努力

- Spriting 合并多张小图为一张大图供浏览器 JS 切割使用
 - 不能区别对待
- Inlining 内联，将图片嵌入到 CSS 或者 HTML 文件中，减少网络请求次数
- Concatenation 拼接，将多个体积较小的 JavaScript 使用 webpack 等工具打包成 1 个体积更大的 JavaScript 文件
 - 1 个文件的改动导致用户重新下载多个文件
- Sharding 分片，将同一页面的资源分散到不同域名下，提升连接上限

HTTP/1.1 不支持服务器推送消息



第 2 课 HTTP/2 有哪些特性?

解决 HTTP/1 性能问题的 HTTP/2

- SPDY (2012-2016)
- HTTP2 (RFC7540, 2015.5)
 - 在应用层上修改，基于并充分挖掘 TCP 协议性能
 - 客户端向 server 发送 request 这种基本模型不会变。
 - 老的 scheme 不会变，没有 http2://。
 - 使用 http/1.x 的客户端和服务端可以无缝的通过代理方式转接到 http/2 上。
 - 不识别 http/2 的代理服务器可以将请求降级到 http/1.x

HTTP

SPDY

SSL

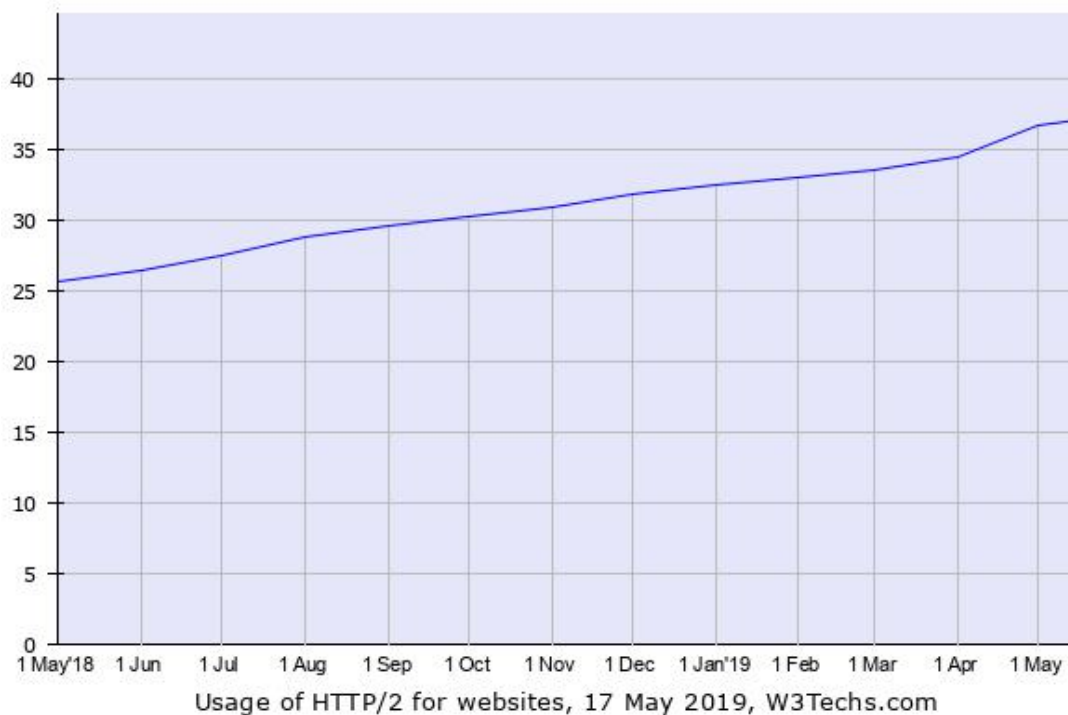
TCP

主流浏览器对 HTTP/2 的支持程度

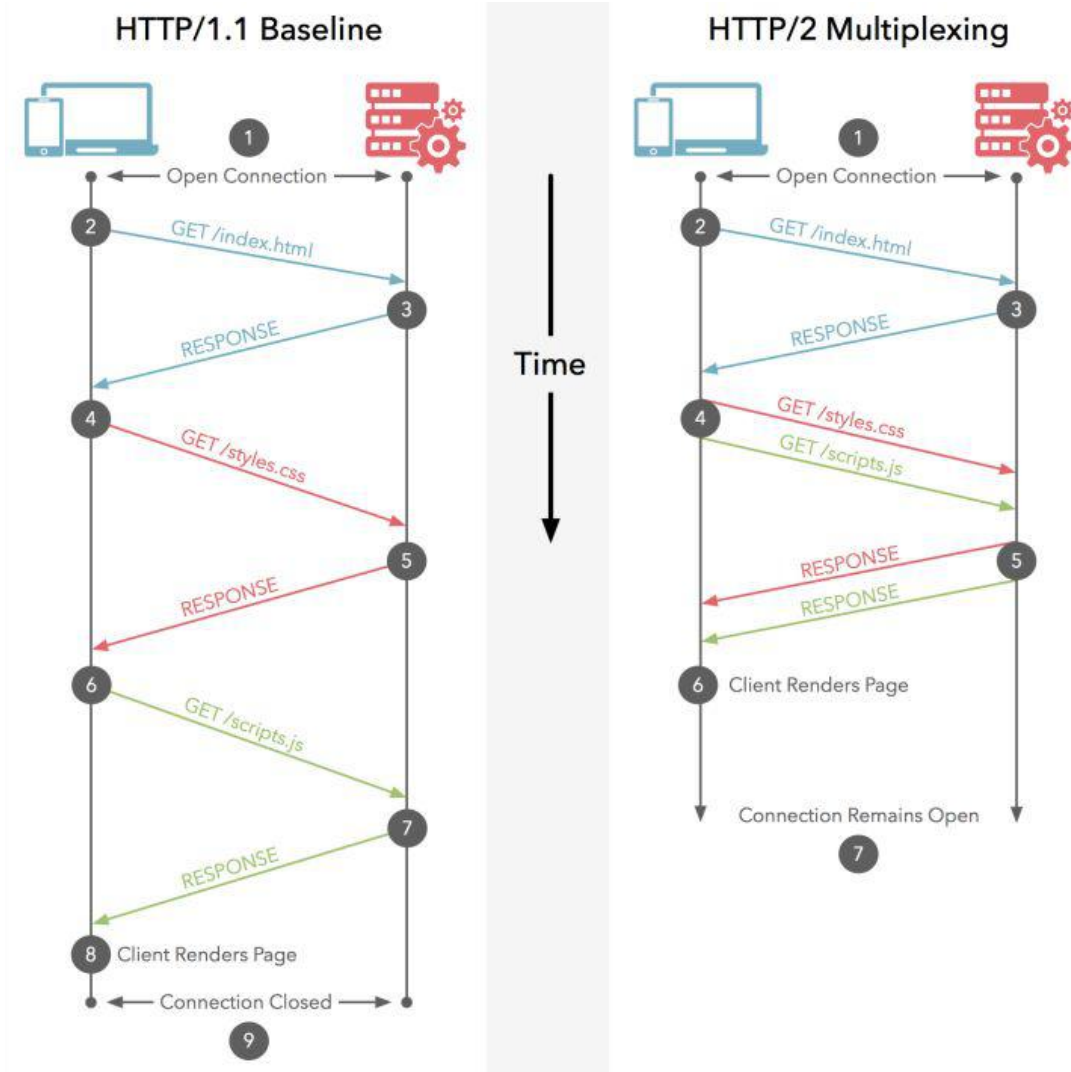
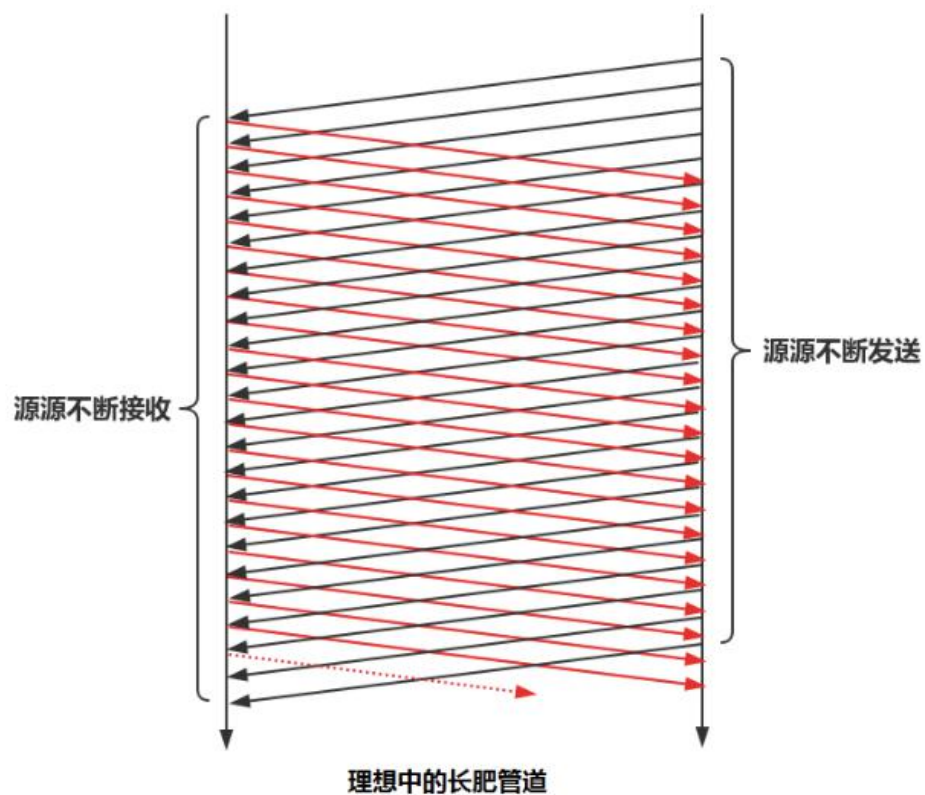
IE	Edge *	Firefox	Chrome	Safari	Opera	iOS Safari *	Opera Mini *	Android Browser *	Chrome for Android
			49						
			51			9.2		4.4	
8	13	47	52			9.3		4.4.4	
11	14	48	53	9.1	39	10	all	52	51
		49	54	10	40				
		50	55	TP	41				
		51	56						

HTTP/2 的应用状况

- 截止 2019.5.17 号，互联网上使用 HTTP/2 协议的站点已经达到 37.2%
- 快速推广的原因
 - 未改变 HTTP/1.1 的语义
 - 基于 TCP，仅在应用层变动



多路复用带来的提升

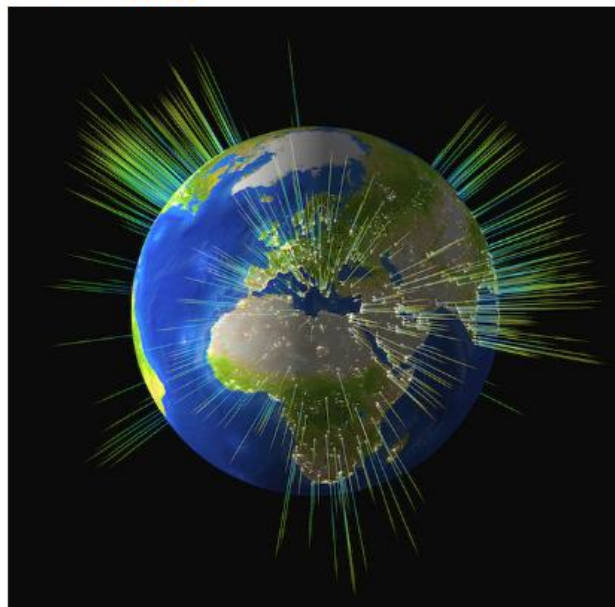


HTTP/2 的强大之处

- <https://http2.akamai.com/demo>

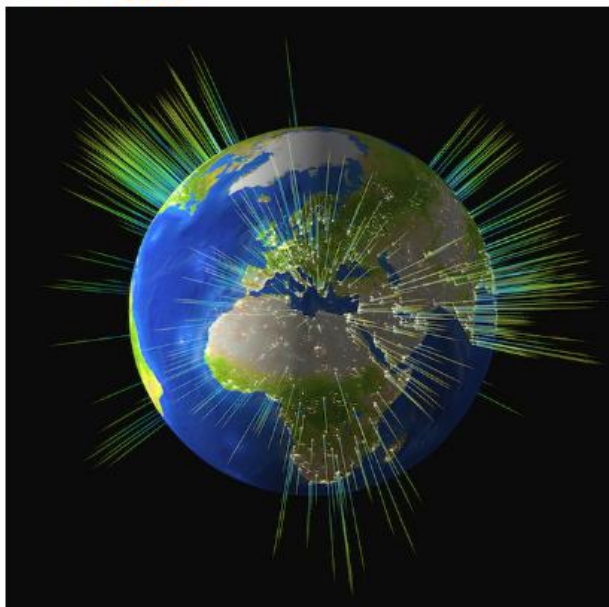
HTTP/1.1

Latency: 36ms
Load time: 14.18s



HTTP/2

Latency: 82ms
Load time: 2.28s



HTTP/2 主要特性

- 传输数据量的大幅减少
 - 以二进制方式传输
 - 标头压缩
- 多路复用及相关功能
 - 消息优先级
- 服务器消息推送
 - 并行推送

第 3 课 如何使用 Wireshark 解密 TLS/SSL 报文?

Chrome 浏览器检测 HTTP/2 插件

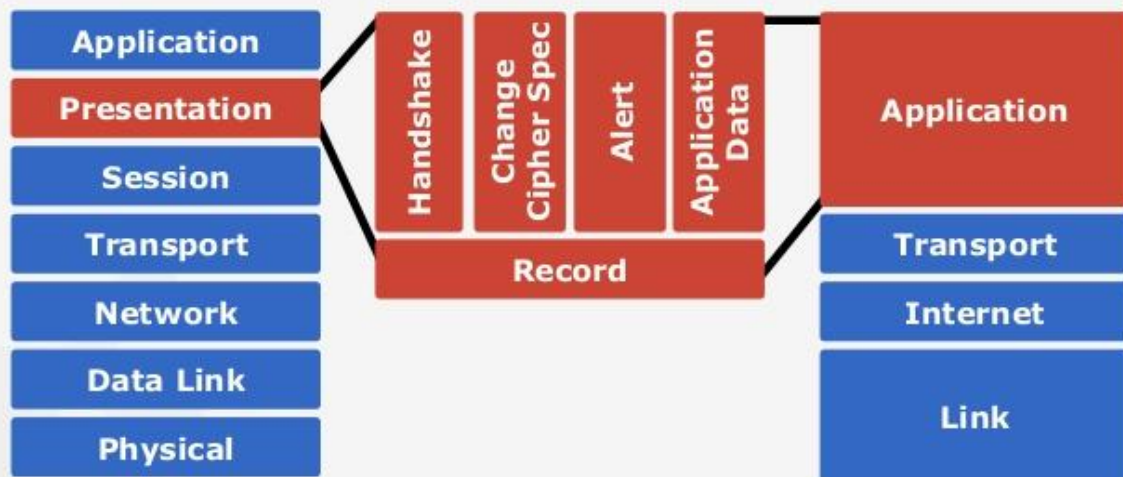
- HTTP/2 and SPDY indicator
 - <https://chrome.google.com/webstore/detail/http2-and-spy-indicator/mpbpobfflnpcgagjjhmgmgnchggcjblin>

在 HTTP/2 应用层协议之下的 TLS 层

SSL/TLS in Common Models

ISO/OSI model

TCP/IP model



TLS1.2 的加密算法

- 常见加密套件



- 对称加密算法: AES_128_GCM

- 每次建立连接后, 加密密钥都不一样

- 密钥生成算法: ECDHE

- 客户端与服务器通过交换部分信息, 各自独立生成最终一致的密钥

Wireshark 如何解密 TLS 消息?

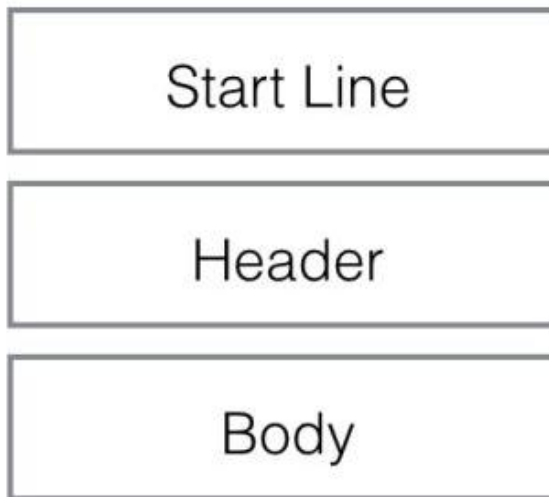
- 原理：获得 TLS 握手阶段生成的密钥
 - 通过 Chrome 浏览器 DEBUG 日志中的握手信息生成密钥
- 步骤
 - 配置 Chrome 输出 DEBUG 日志
 - 配置环境变量 SSLKEYLOGFILE
 - 在 Wireshark 中配置解析 DEBUG 日志
 - 编辑->首选项->Protocols->TLS/SSL
 - (Pre)-Master-Secret log filename

```
CLIENT_RANDOM afc4de62f9507f5783
ecf4bd78fd429c6ca6077a70afe94bad
7d
CLIENT_RANDOM 524de51b66f0d1d6e6
82fbc3e7fbf214cb408a4ded1f19aaf4
ea
CLIENT_HANDSHAKE_TRAFFIC_SECRET
d26d9369722978831a0d08ca0834878c
201e5a227b742b9ef41e0dee064b9f5c
SERVER_HANDSHAKE_TRAFFIC_SECRET
d26d9369722978831a0d08ca0834878c
b957b59ad738d46652723600113e3df1
```

二进制格式与可见性

- TLS/SSL 降低了可见性门槛
 - 代理服务器没有私钥不能看到内容

HTTP/1.x



HTTP/2



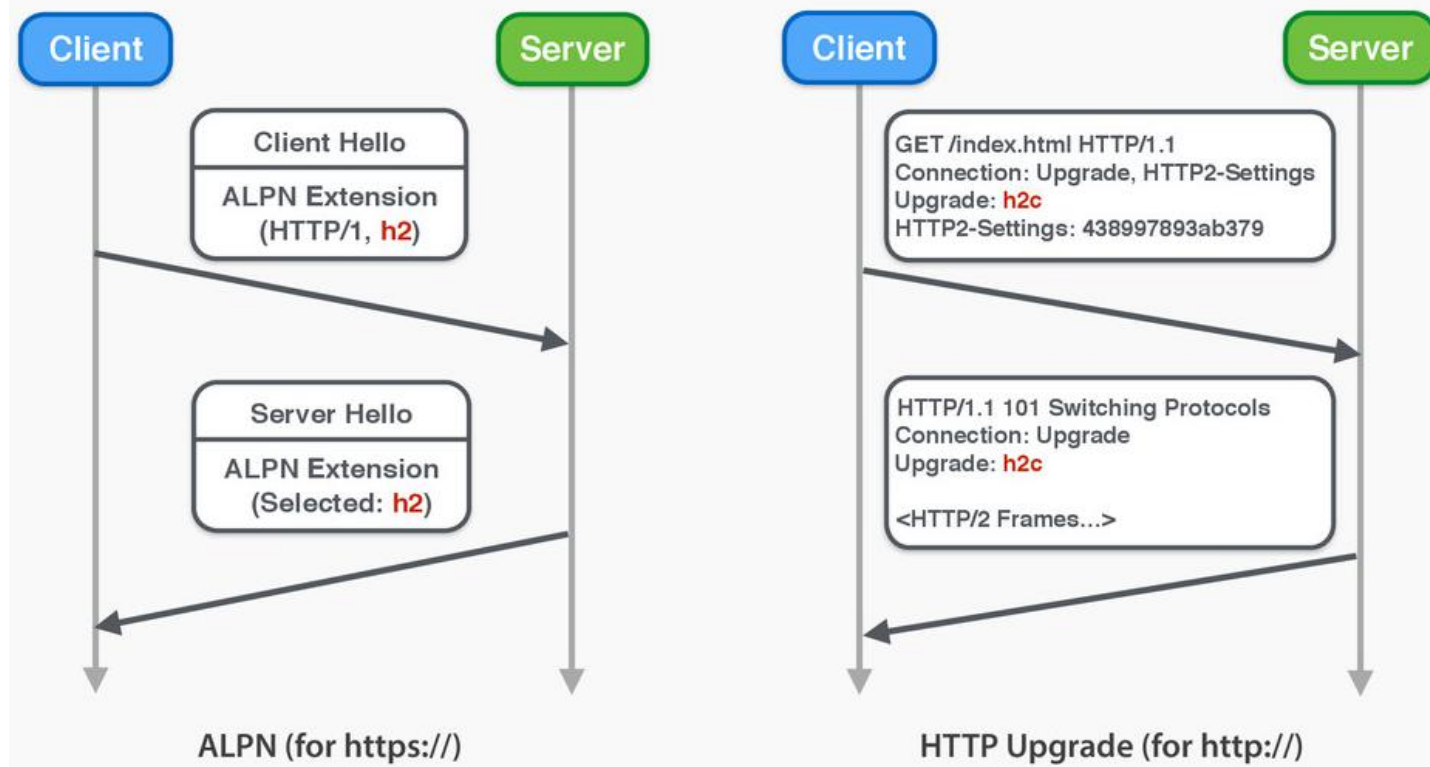
第 4 课 h2c: 如何从 http://升级到 HTTP/2 协议?

HTTP/2 是不是必须基于 TLS/SSL 协议?

- IETF 标准不要求必须基于TLS/SSL协议
- 浏览器要求必须基于TLS/SSL协议
- 在 TLS 层 ALPN (Application Layer Protocol Negotiation)扩展做协商, 只认 HTTP/1.x 的代理服务器不会干扰 HTTP/2
- shema: http://和 https:// 默认基于 80 和 443 端口
- h2: 基于 TLS 协议运行的 HTTP/2 被称为 h2
- h2c: 直接在 TCP 协议之上运行的 HTTP/2 被称为 h2c

h2 与 h2c

Protocol Negotiation



H2C: 不使用 TLS 协议进行协议升级 (1)

- 客户端测试工具: curl (7.46.0版本)
 - curl http://nghttp/2.org -http/2 -v

```
GET / HTTP/1.1\r\n
Host: nghttp2.org\r\n
User-Agent: curl/7.46.0\r\n
Accept: */*\r\n
Connection: Upgrade, HTTP2-Settings\r\n
Upgrade: h2c\r\n
HTTP2-Settings: AAMAAABkAAQAAP__\r\n
\r\n
```

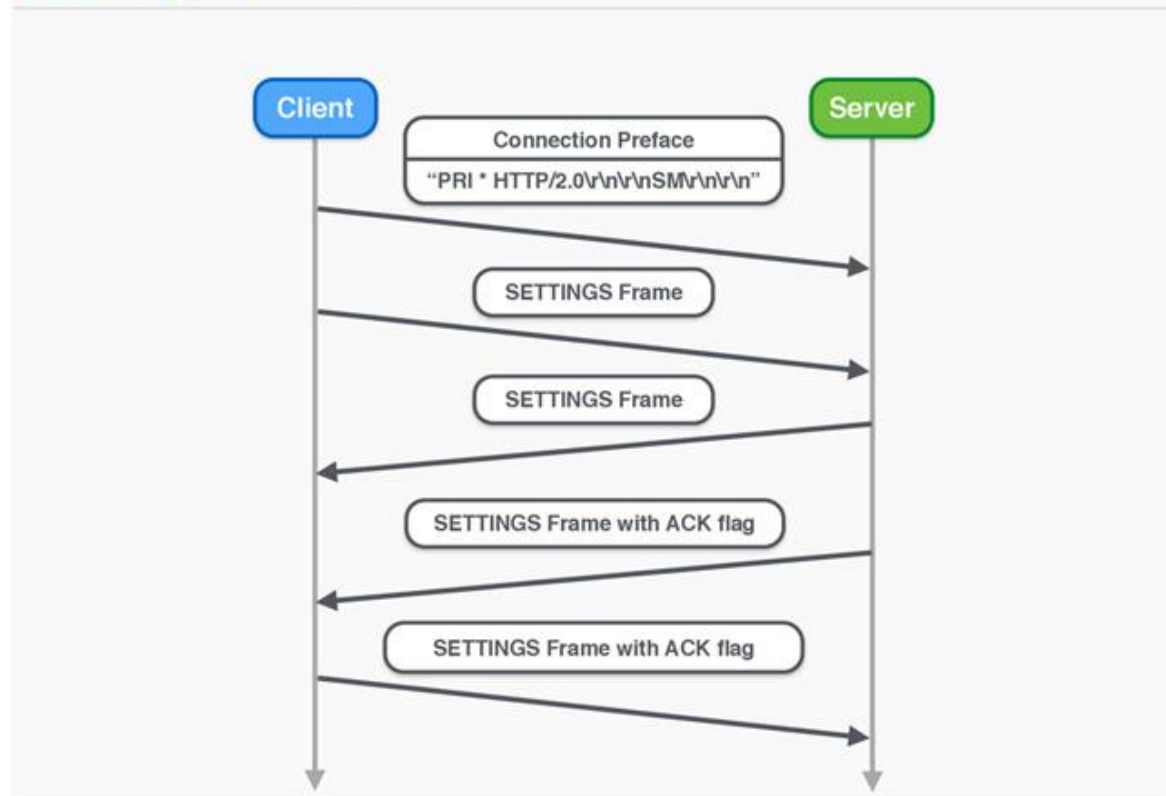
```
HTTP/1.1 101 Switching Protocols\r\n
Connection: Upgrade\r\n
Upgrade: h2c\r\n
\r\n
```

H2C: 客户端发送的 Magic 帧

- Preface (ASCII 编码, 12字节)
 - 何时发送?
 - 接收到服务器发送来的 101 Switching Protocols
 - TLS 握手成功后
 - Preface 内容
 - 0x505249202a20485454502f322e300d0a0d0a534d0d0a0d0a
 - PRI * HTTP/2.0\r\n\r\nSM\r\n\r\n
 - 发送完毕后, 应紧跟 SETTING 帧

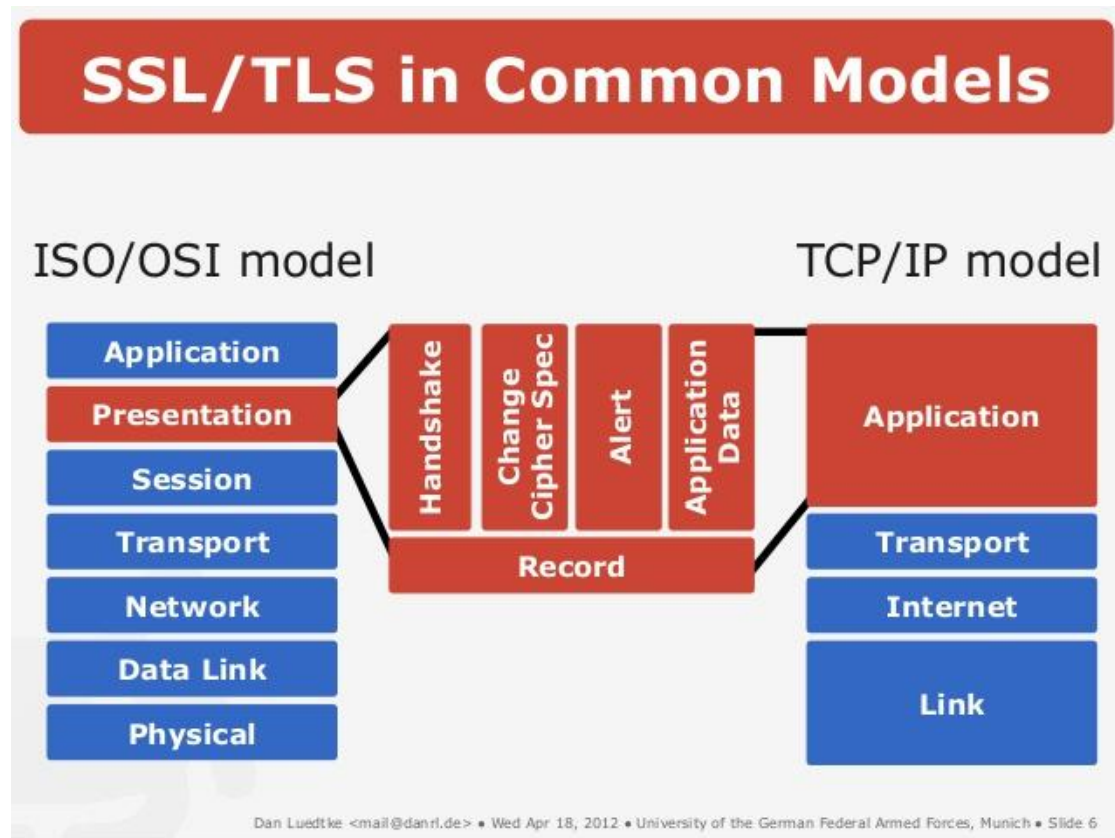
统一的连接过程

Starting HTTP/2



第 5 课 h2: 如何从 https://升级到 HTTP/2 协议?

在 HTTP/2 应用层协议之下的 TLS 层



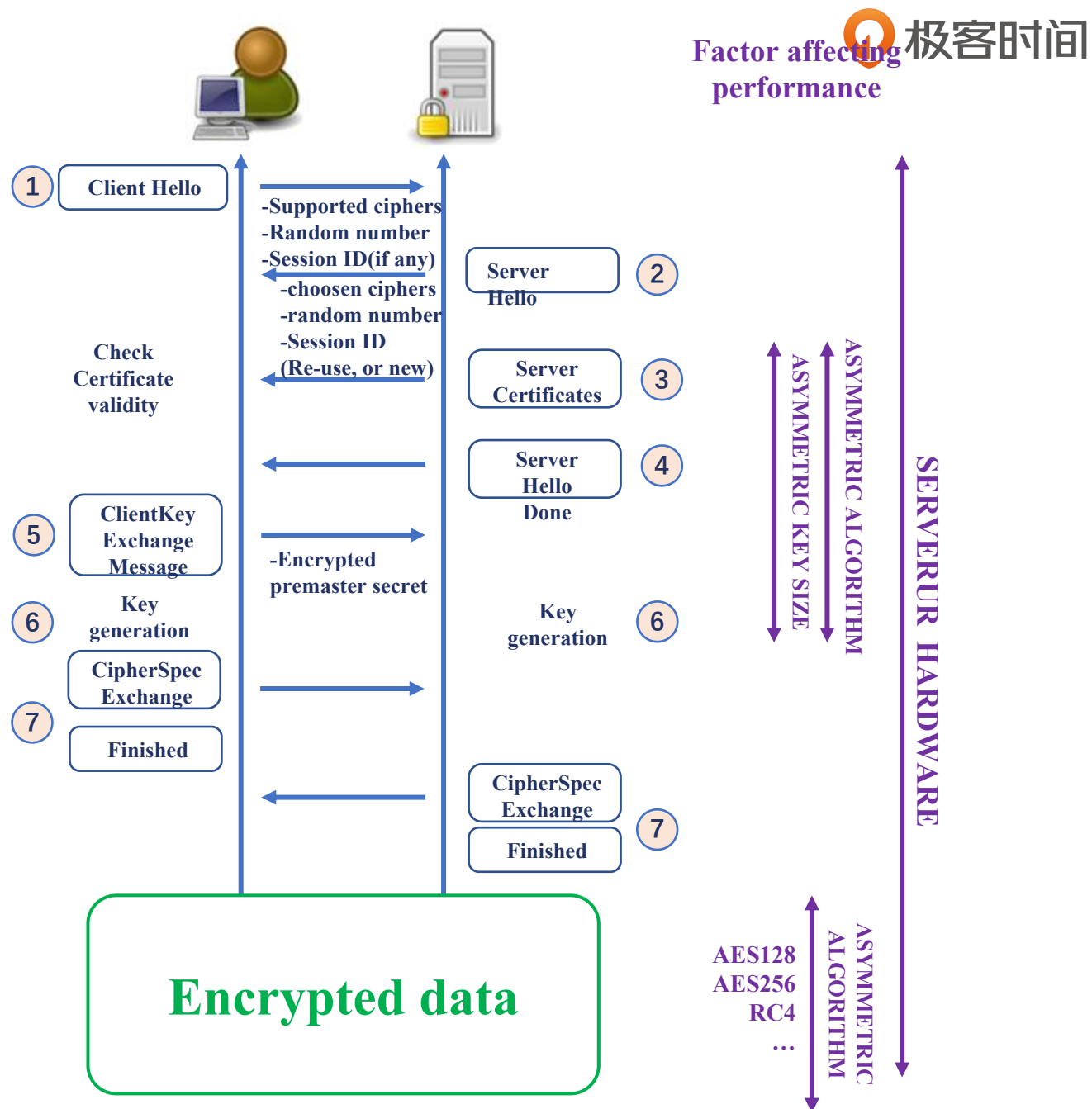
TLS 通讯过程

验证身份

达成安全套件共识

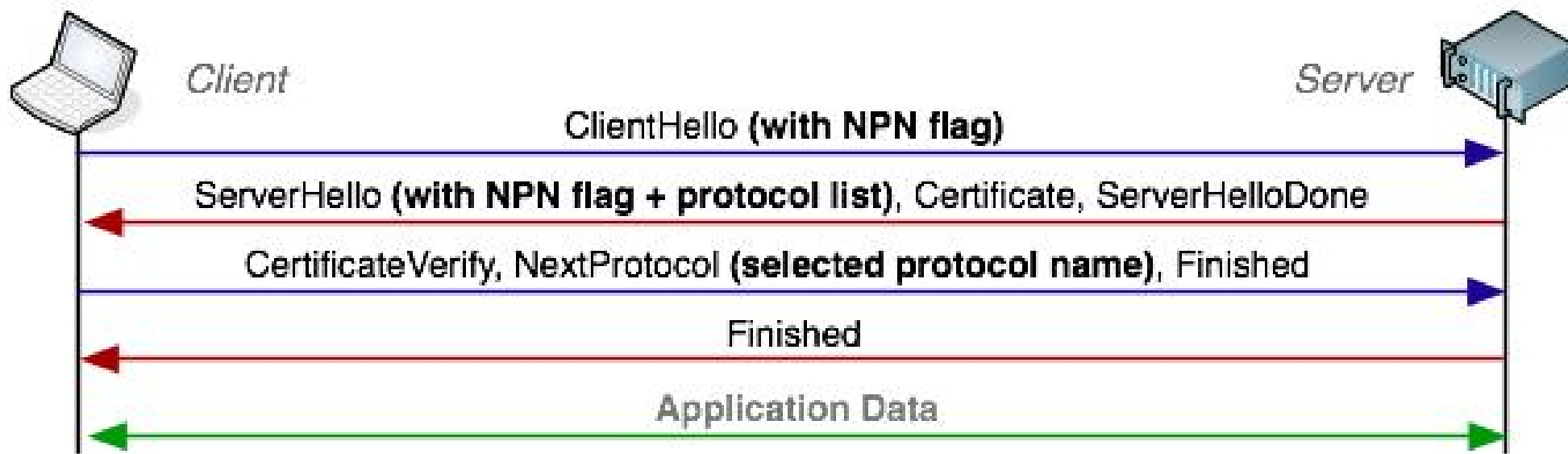
传递密钥

加密通讯



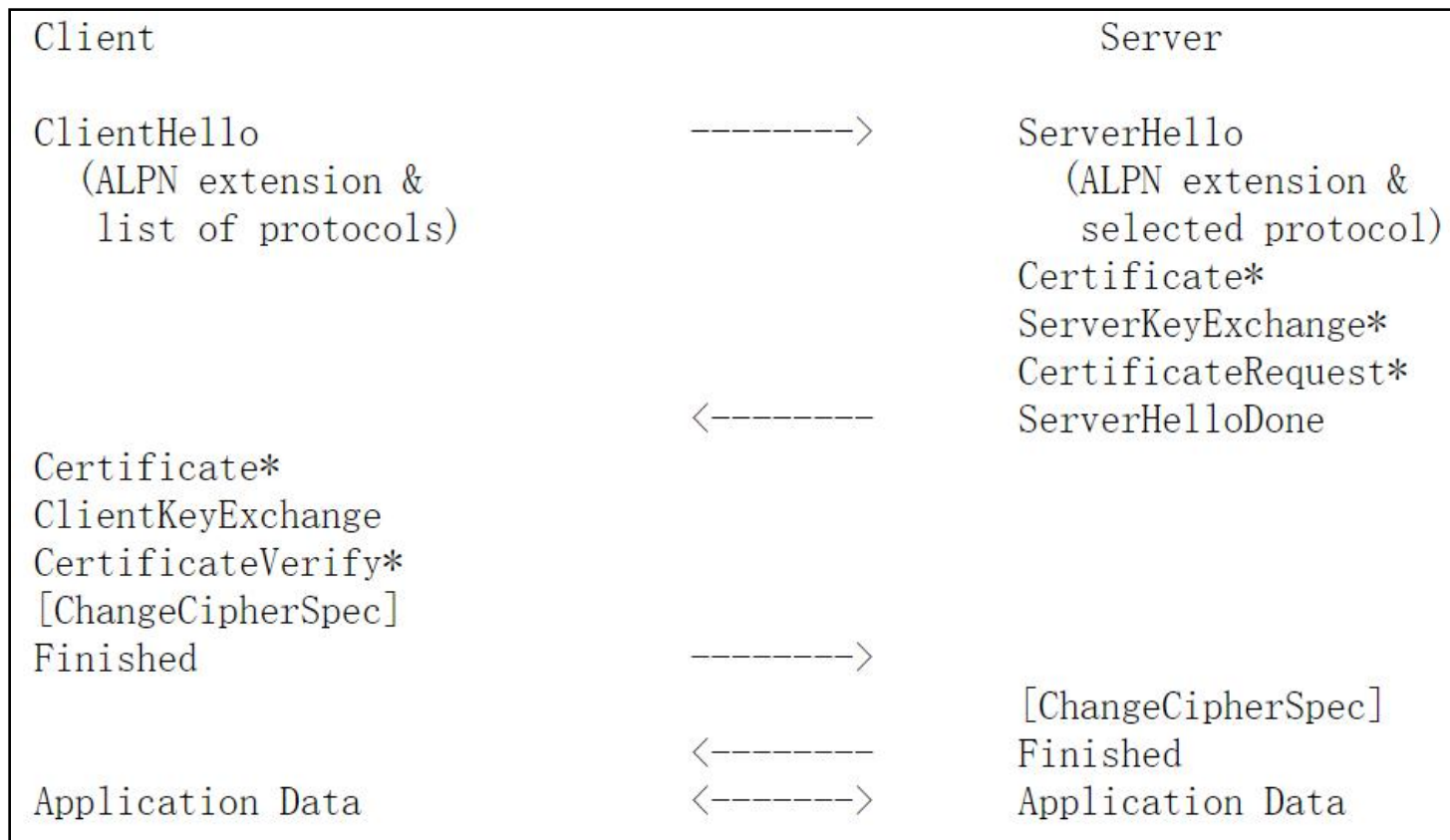
Next Protocol Negotiation (NPN)

- SPDY 使用的由客户端选择协议的 NPN 扩展



Application-Layer Protocol Negotiation Extension

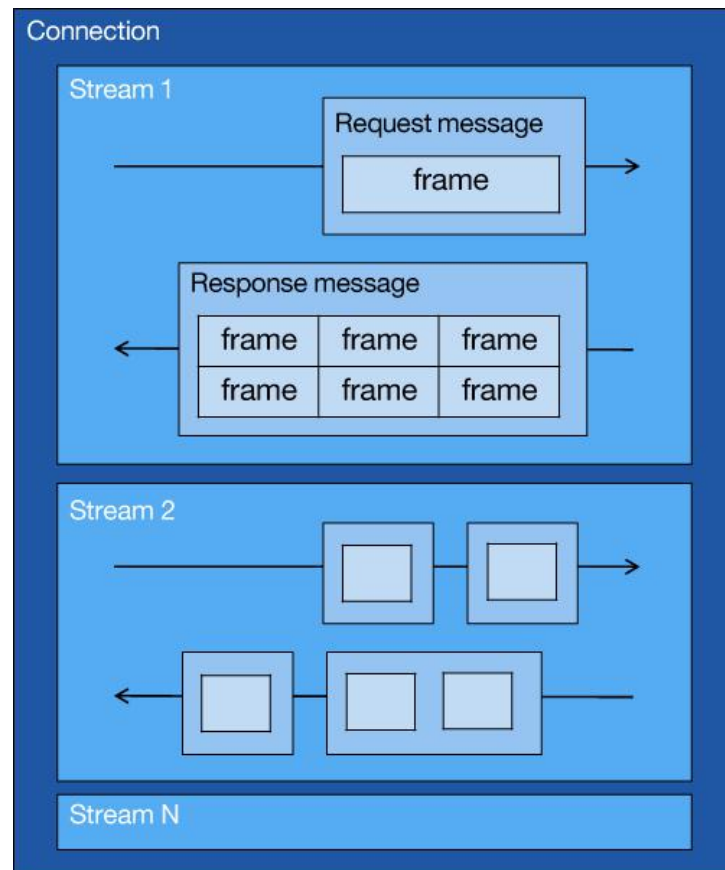
- RFC7301



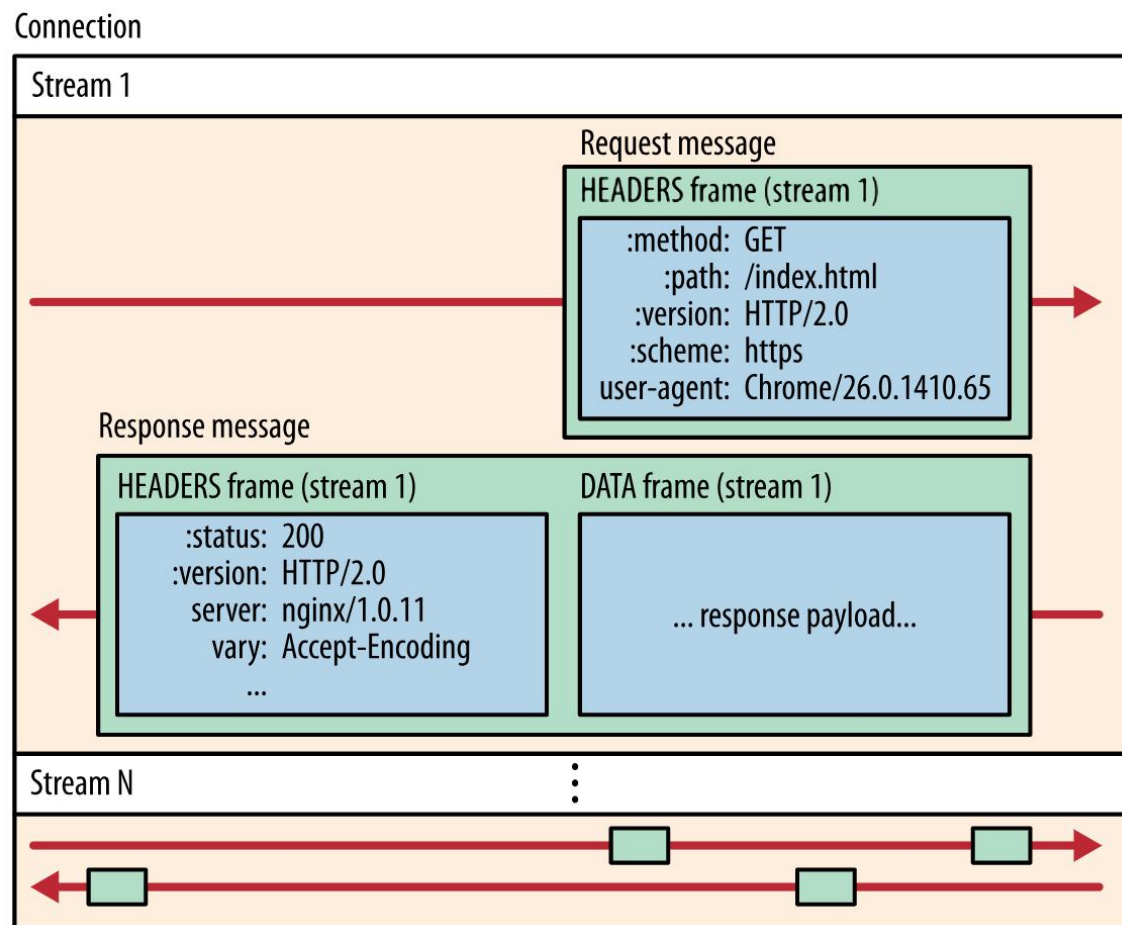
第 6 课 Stream、Message、Frame 间的关系

HTTP/2 核心概念

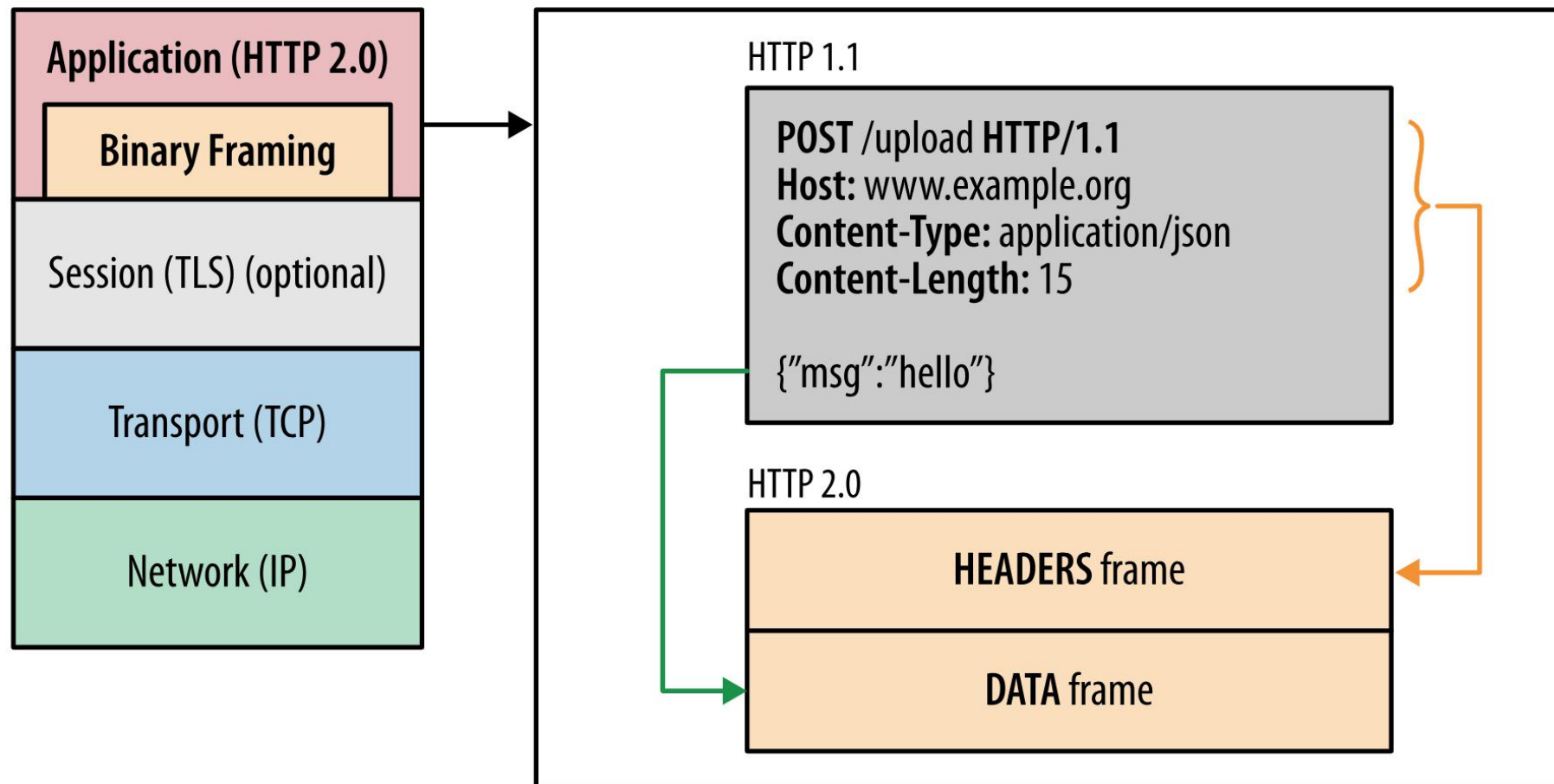
- 连接 Connection: 1个 TCP 连接, 包含一个或者多个 Stream
- 数据流 Stream: 一个双向通讯数据流, 包含 1 条或者多条 Message
- 消息 Message: 对应 HTTP/1 中的请求或者响应, 包含一条或者多条 Frame
- 数据帧 Frame: 最小单位, 以二进制压缩格式存放 HTTP/1 中的内容



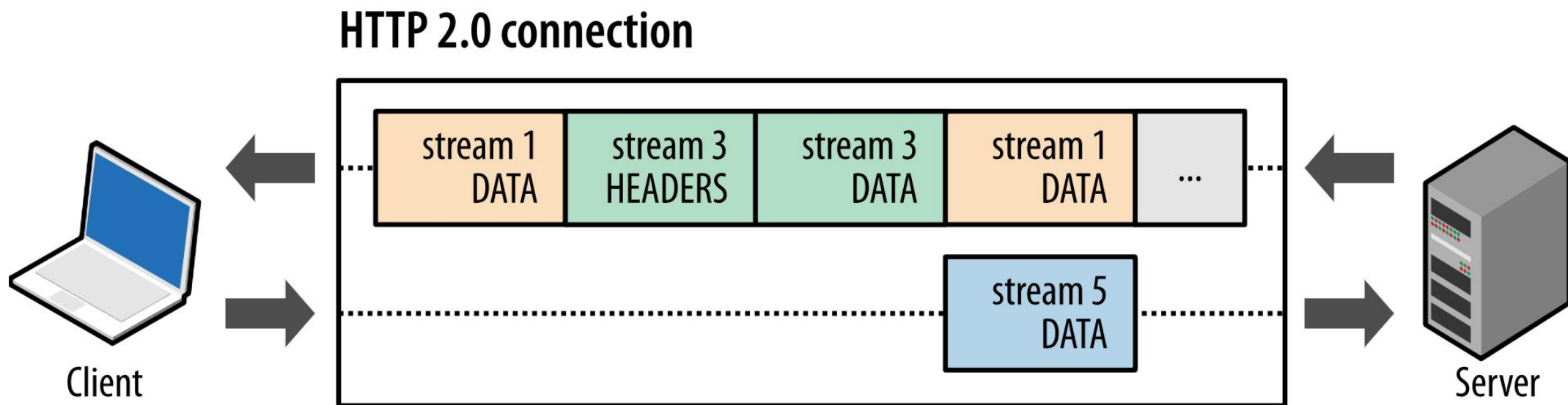
Stream、Message、Frame 间的关系



消息的组成：HEADERS 帧与 DATA 帧

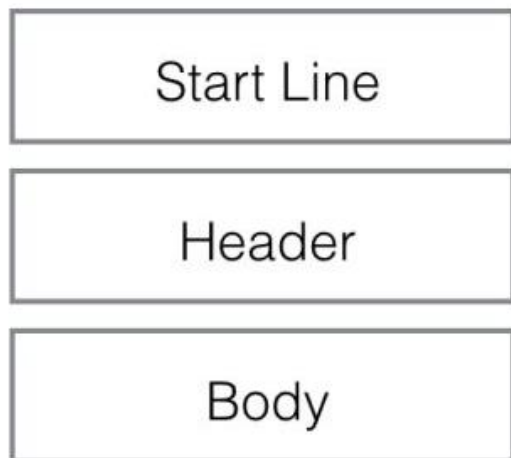


传输中无序，接收时组装



消息与帧

HTTP/1.x

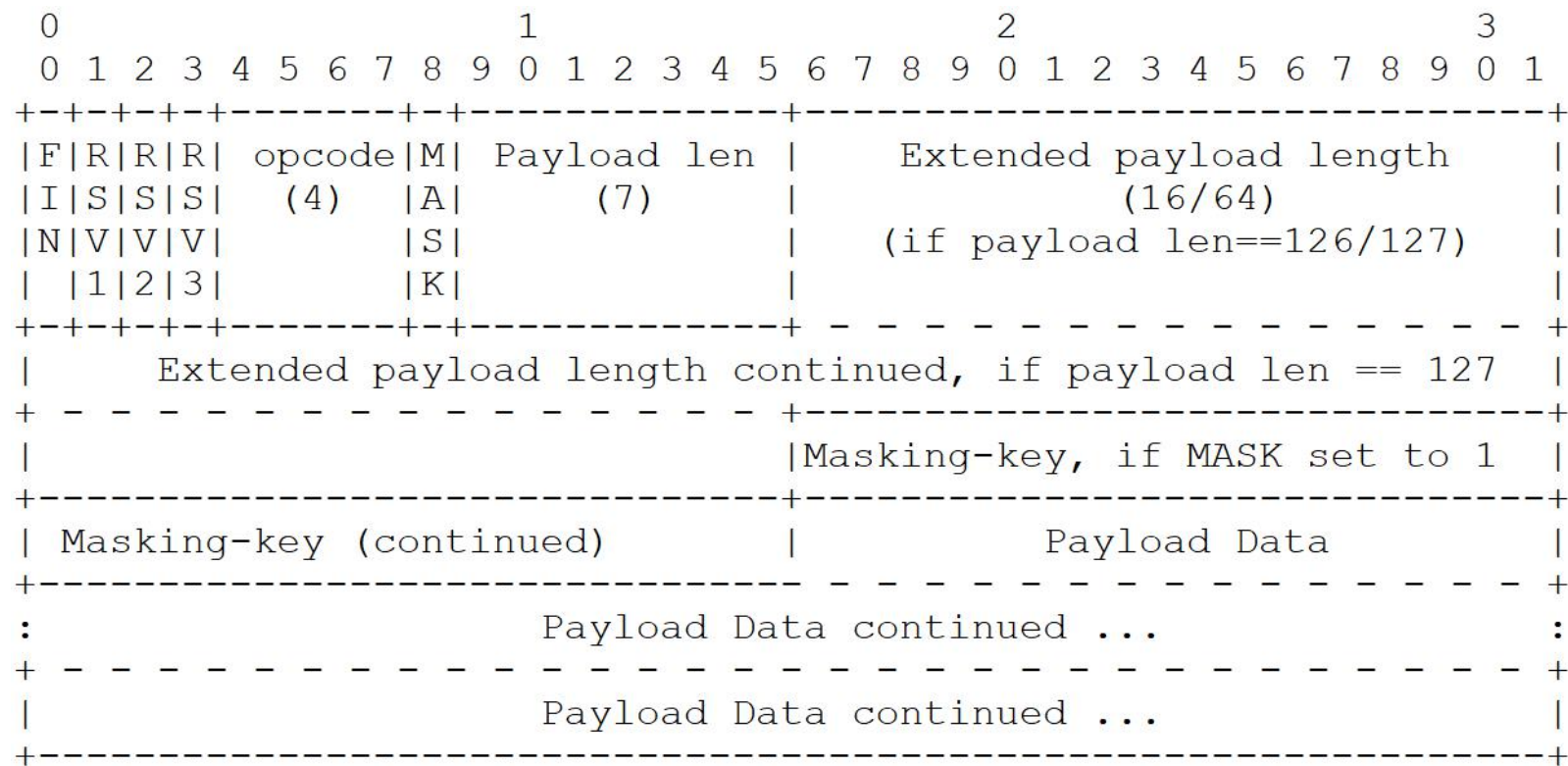


HTTP/2

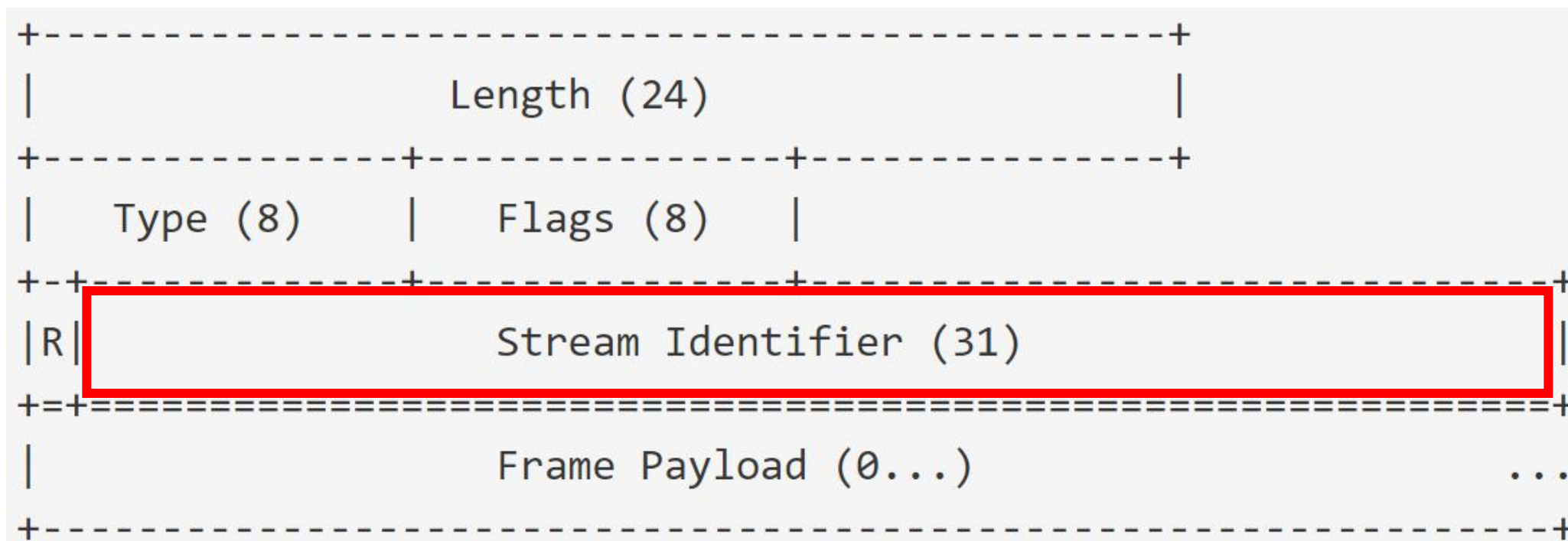


第 7 课 帧格式：Stream 流 ID 的作用

对比WebSocket的帧格式



9 字节标准帧头部



标准帧头示意

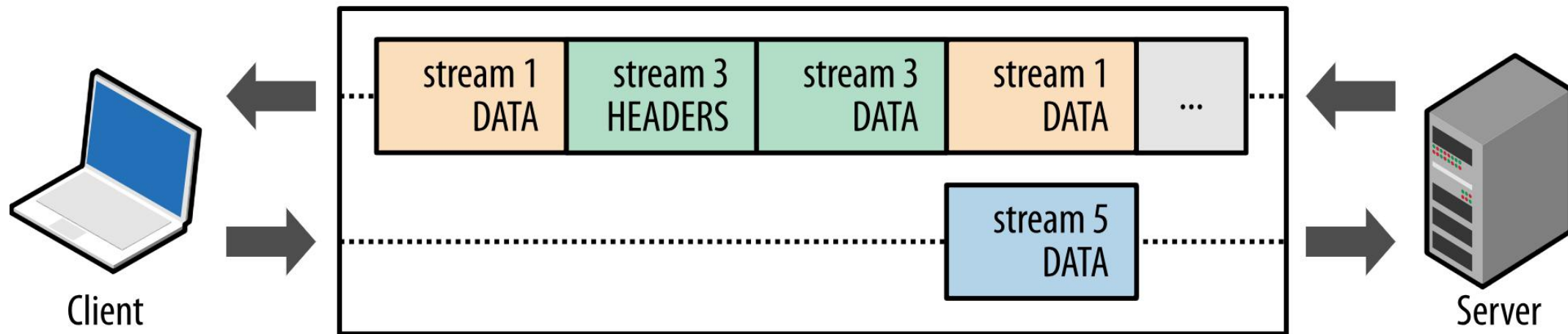


```
▼ HyperText Transfer Protocol 2
  ▼ Stream: HEADERS, Stream ID: 1, Length 198, 200 OK
    Length: 198
    Type: HEADERS (1)
    > Flags: 0x04
    0... .. = Reserved: 0x0
    .000 0000 0000 0000 0000 0000 0000 0001 = Stream Identifier: 1
    [Pad Length: 0]
    Header Block Fragment: 886196c361be940baa681fa50400bea01ab8005c1
    [Header Length: 401]
    [Header Count: 13]
    > Header: :status: 200 OK
    > Header: date: Fri, 17 May 2019 04:00:23 GMT
    > Header: content-type: text/html
    > Header: last-modified: Thu, 18 Apr 2019 06:19:33 GMT
    > Header: etag: "5cb816f5-19d8"
    > Header: accept-ranges: bytes
    > Header: content-length: 6616
    > Header: x-backend-header-rtt: 0.024989
    .....
```

Stream ID 的作用 (1)

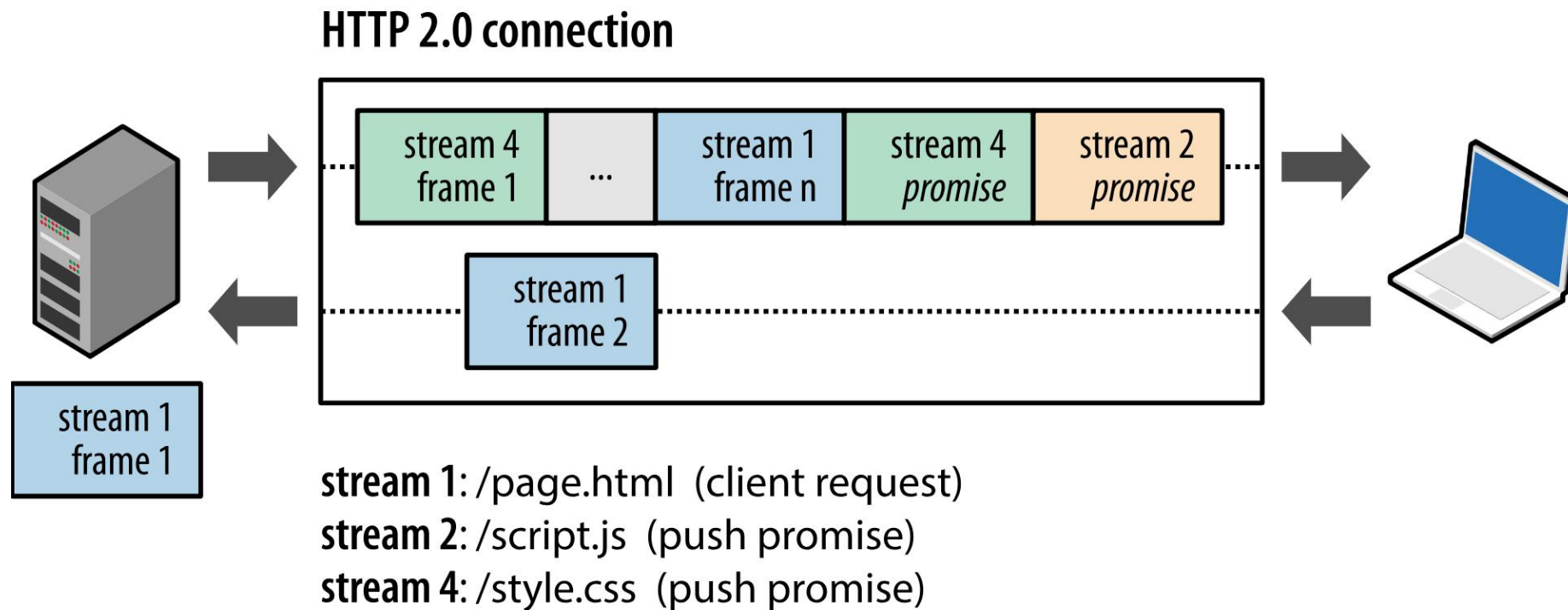
- 实现多路复用的关键
 - 接收端的实现可据此并发组装消息
 - 同一 Stream 内的 frame 必须是有序的（无法并发）
 - SETTINGS_MAX_CONCURRENT_STREAMS 控制着并发 Stream 数

HTTP 2.0 connection



Stream ID 的作用 (2)

- 推送依赖性请求的关键
 - 由客户端建立的流必须是奇数
 - 由服务器建立的流必须是偶数



Stream ID 的作用 (3)

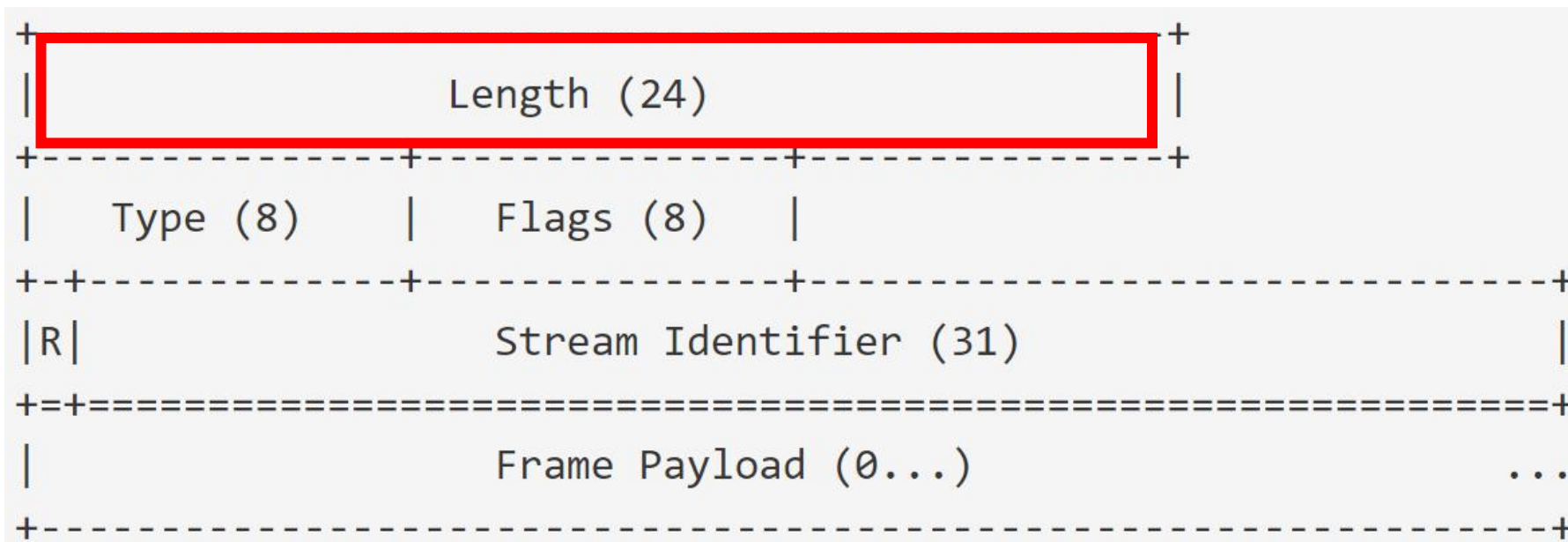
- 流状态管理的约束性规定
 - 新建立的流 ID 必须大于曾经建立过的状态为 opened 或者 reserved 的流 ID
 - 在新建立的流上发送帧时，意味着将更小 ID 且为 idle 状态的流置为 closed 状态
 - Stream ID 不能复用，长连接耗尽 ID 应创建新连接

Stream ID 的作用 (4)

- 应用层流控仅影响数据帧
 - Stream ID 为 0 的流仅用于传输控制帧
- 在HTTP/1 升级到 h2c 中, 以 ID 为 1 流返回响应, 之后流进入half-closed (local)状态

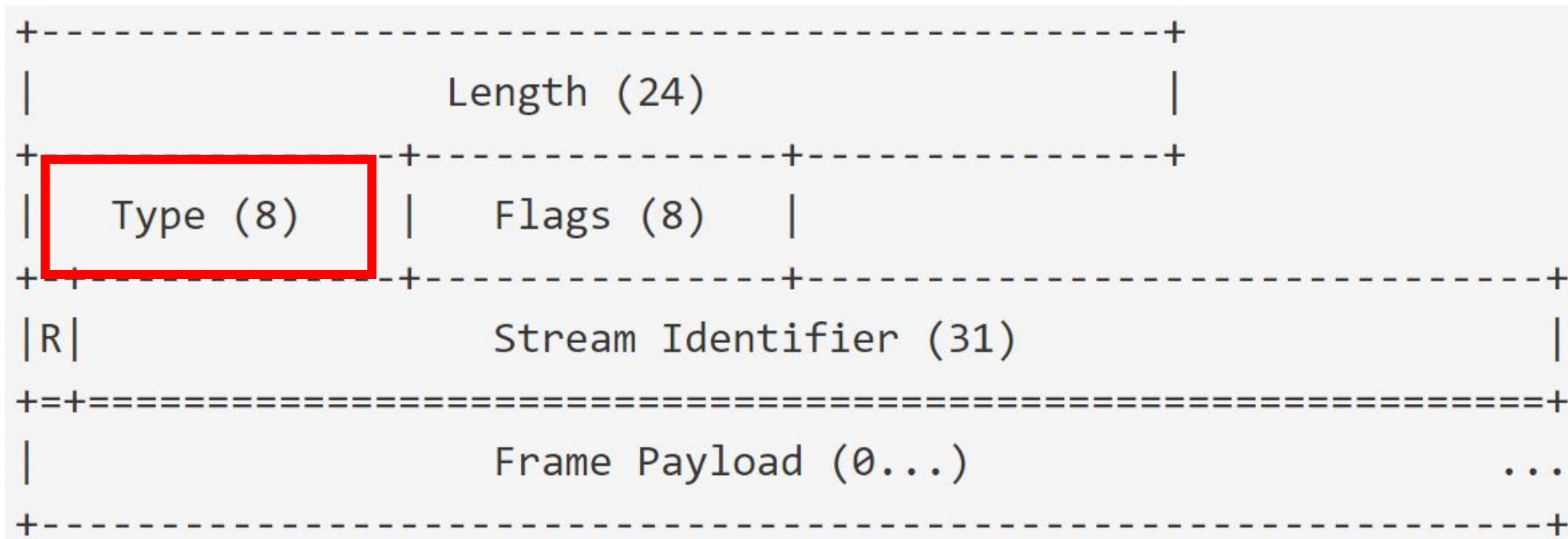
第 8 课 帧格式：帧类型及设置帧的子类型

9 字节标准帧头部：帧长度



- 0 至 2^{14} (16,384) -1
 - 所有实现必须可以支持 16KB 以下的帧
- 2^{14} (16,384) 至 $2^{24}-1$ (16,777,215)
 - 传递 16KB 到 16MB 的帧时，必须接收端首先公布自己可以处理此大小
 - 通过 SETTINGS_MAX_FRAME_SIZE 帧 (Identifier=5) 告知

帧类型 Type

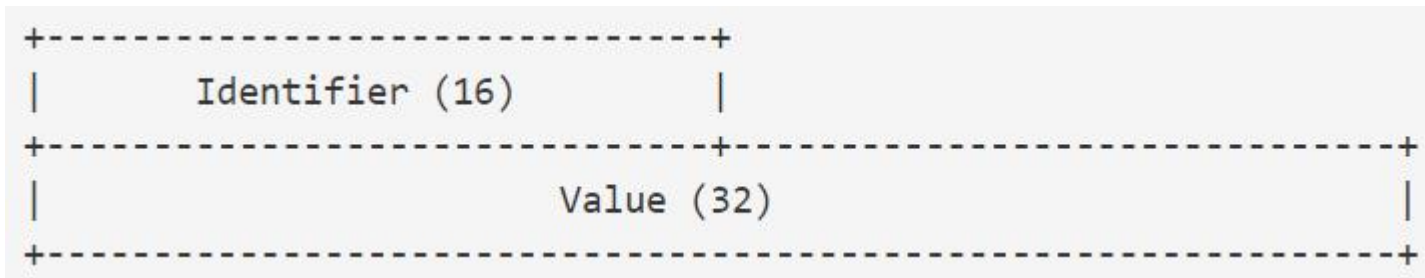


帧类型

帧类型	类型编码	用途
DATA	0x0	传递HTTP包体
HEADERS	0x1	传递HTTP头部
PRIORITY	0x2	指定Stream流的优先级
RST_STREAM	0x3	终止Stream流
SETTINGS	0x4	修改连接或者Stream流的配置
PUSH_PROMISE	0x5	服务端推送资源时描述请求的帧
PING	0x6	心跳检测，兼具计算RTT往返时间的功能
GOAWAY	0x7	优雅的终止连接或者通知错误
WINDOW_UPDATE	0x8	实现流量控制
CONTINUATION	0x9	传递较大HTTP头部时的持续帧

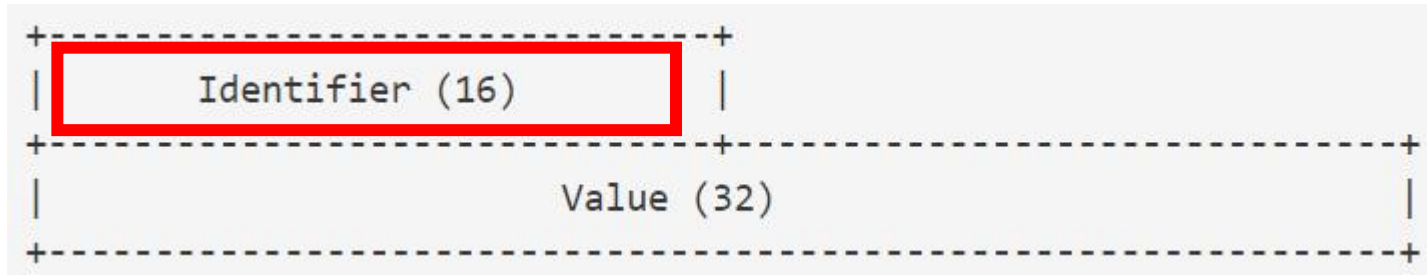
Setting 设置帧格式 (type=0x4)

- 设置帧并不是“协商”，而是发送方向接收方通知其特性、能力
- 一个设置帧可同时设置多个对象



- Identifier: 设置对象
- Value: 设置值

Setting 设置对象的类型



- 设置类型

- SETTINGS_HEADER_TABLE_SIZE (0x1): 通知对端索引表的最大尺寸（单位字节，初始 4096 字节）
- SETTINGS_ENABLE_PUSH (0x2): Value 设置为 0 时可禁用服务器推送功能，1 表示启用推送功能
- SETTINGS_MAX_CONCURRENT_STREAMS (0x3): 告诉接收端允许的最大并发流数量
- SETTINGS_INITIAL_WINDOW_SIZE (0x4): 声明发送端的窗口大小，用于Stream级别流控，初始值 $2^{16}-1$ (65,535) 字节
- SETTINGS_MAX_FRAME_SIZE (0x5): 设置帧的最大大小，初始值 2^{14} (16,384) 字节
- SETTINGS_MAX_HEADER_LIST_SIZE (0x6): 知会对端头部索引表的最大尺寸，单位字节，基于未压缩前的头部

第 9 课 HPACK 如何减少 HTTP 头部的大小?

HPACK 头部压缩

- RFC7541
- 三种压缩方式
 - 静态字典
 - 动态字典
 - 压缩算法：Huffman 编码（最高压缩比 8:5）

静态字典

- <https://http2.github.io/http2-spec/compression.html#static.table.definition>
- 含有 value 的表项
- 不含有 value 的表项

Index	Header Name	Header Value
1	:authority	
2	:method	GET
3	:method	POST
4	:path	/
5	:path	/index.html
6	:scheme	http
7	:scheme	https
8	:status	200
...	...	...
32	cookie	
...	...	...
60	via	
61	www-authenticate	

HPACK 压缩示意

- 同一个索引空间的 HEADER 表

Request headers

:method	GET
:scheme	https
:host	example.com
:path	/resource
user-agent	Mozilla/5.0 ...
custom-hdr	some-value



Static table

1	:authority	
2	:method	GET
...	...	...
51	referer	
...	...	...
62	user-agent	Mozilla/5.0 ...
63	:host	example.com
...	...	...

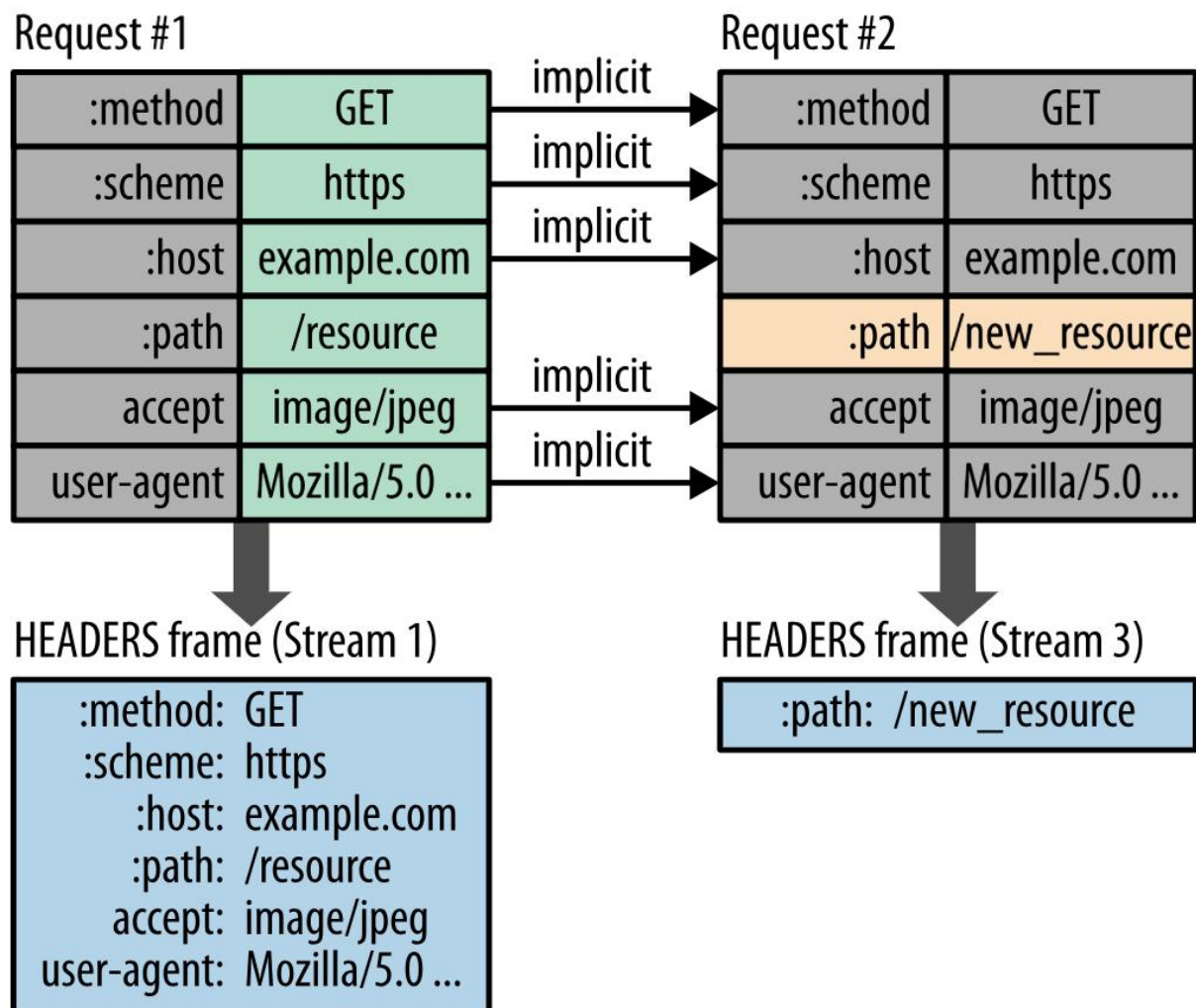
Dynamic table



Encoded headers

2	
7	
63	
19	Huffman("/resource")
62	
	Huffman("custom-hdr")
	Huffman("some-value")

索引表用法示意



HPACK 压缩比: h2load

```
[root@iZbp1eocwx9cd067a15x0Z web]# h2load https://blog.cloudflare.com -n 4
starting benchmark...
spawning thread #0: 1 total client(s). 4 total requests
TLS Protocol: TLSv1.2
Cipher: ECDHE-ECDSA-AES128-GCM-SHA256
Server Temp Key: ECDH P-256 256 bits
Application protocol: h2
progress: 25% done
progress: 50% done
progress: 75% done
progress: 100% done

finished in 2.41s, 1.66 req/s, 44.89KB/s
requests: 4 total, 4 started, 4 done, 4 succeeded, 0 failed, 0 errored, 0 timeout
status codes: 4 2xx, 0 3xx, 0 4xx, 0 5xx
traffic: 107.98KB (110575) total, 918B (918) headers (space savings 61.94%), 106.83KB (109392) data
```

	min	max	mean	sd	+/- sd
time for request:	449.59ms	591.08ms	492.22ms	66.28ms	75.00%
time for connect:	436.25ms	436.25ms	436.25ms	0us	100.00%
time to 1st byte:	886.63ms	886.63ms	886.63ms	0us	100.00%
req/s :	1.66	1.66	1.66	0.00	100.00%

第 10 课 HPACK 中如何使用 Huffman 树编码?

Huffman 编码

- 原理：出现概率较大的符号采用较短的编码，概率较小的符号采用较长的编码
- 静态 Huffman 编码
 - <https://httpwg.org/specs/rfc7541.html#huffman.code>
- 动态 Huffman 编码

Huffman 树的构造过程

1. 计算各字母的出现概率
2. 将出现频率最小的两个字母相加构成子树，左小右大
3. 重复步骤 2，直至完成树的构造
4. 给树的左链接编码为 0，右链接编码为 1
5. 每个字母的编码即从根结点至所在叶结点中所有链接的编码

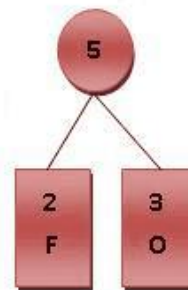
Huffman 树构造过程 (1)

- 有 FORGET 这六个字母需要编码，其出现频率如下

Symbol	F	O	R	G	E	T
Frequency	2	3	4	4	5	7

Huffman 树构造过程 (2)

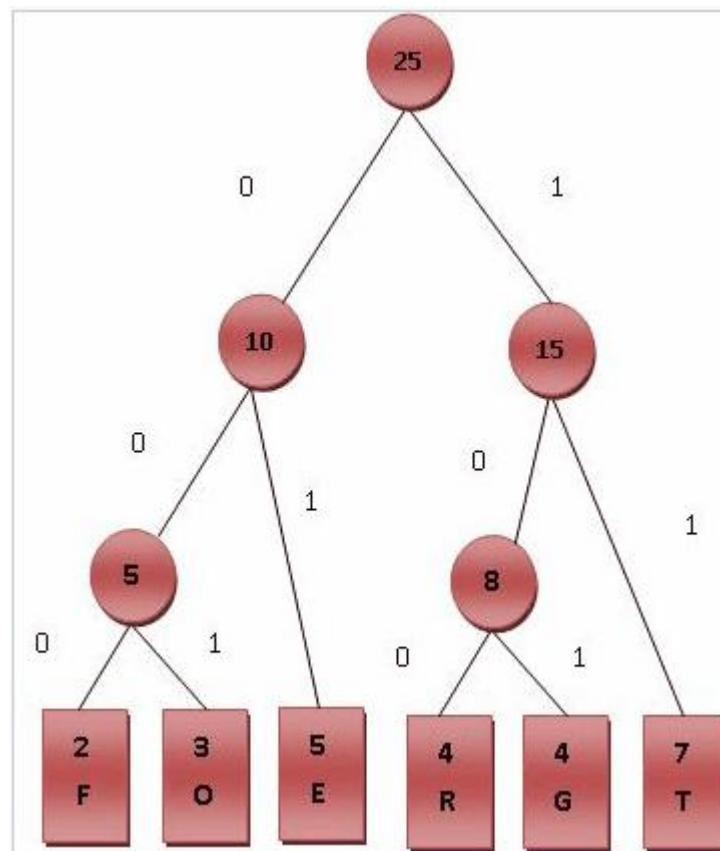
1. 将概率为 2 的 F 与概率为 3 的 O 构成子树，其父结点概率为 5
2. 将概率为 4 的 R 与概率为 4 的 G 构成子树，其父结点概率为 8
3. 将概率为 5 的 E 与第 1 步生成的概率为 5 的子树合并
4. 将概率为 7 的 T 与第 2 步生成的概率为 8 的子树合并
5. 将第 3 步生成的概率为 10 的子树与第 4 步生成的概率为 15 的子树合并



Huffman 树构造过程

- 最终编码如下

Symbol	F	O	R	G	E	T
Frequency	2	3	4	4	5	7
CODE	000	001	100	101	01	11



Huffman 编码举例

- if-none-match "5cb816f5-19d8"
- Huffman 码
 - 11111100101101100100100011011110000010111001001010110110101100000101111100100011110111111001
- 69 8c fe 5b 24 6f 05 c9 5b 58 2f c8 f7 f3

符号	ASCII码	Huffman码	16进制	位长度
"	34	11111110 01	3f9	10
5	53	011011	1b	6
c	99	00100	4	5
b	98	100011	23	6
8	56	011110	1e	6
1	49	00001	1	5
6	54	011100	1c	6
f	102	100101	25	6
-	45	010110	16	6
9	57	011111	1f	6
d	100	100100	24	6

第 11 课 HPACK 中整型数字的编码

使用 5 位前缀：对小于 31 的整型数字的编码

- 当数字小于 31 (2^5-1) 时的编码

0	1	2	3	4	5	6	7
+	+	+	+	+	+	+	+
?	?	?		Value			
+	+	+	+	+	+	+	+

- 例如：对 10 进行编码

0	1	2	3	4	5	6	7
+	+	+	+	+	+	+	+
x	x	x	0	1	0	1	0
+	+	+	+	+	+	+	+

使用 5 位前缀：对整型大数 1337 的编码举例

1337 is greater than 31.

The 5-bit prefix is filled with its max value (31).

$$I = 1337 - 31 = 1306.$$

1 (1306) is greater than or equal to 128, so the while loop body executes:

$$1 \% 128 == 26$$
$$26 + 128 == 154$$

154 is encoded in 8 bits as: 10011010

l is set to 10 ($1306 / 128 == 10$)

l is no longer greater than or equal to 128, so the while loop terminates.

I, now 10, is encoded in 8 bits as: 00001010.

The process ends.

```

    0    1    2    3    4    5    6    7
+---+---+---+---+---+---+---+---+
| X | X | X | 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 | 1 | 0 | 1 | 0 |
| 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 |
+---+---+---+---+---+---+---+
Prefix = 31, I = 1306
1306 >= 128, encode(154), I = 1306 / 128
10 < 128, encode(10), done

```


第 12 课 HPACK 中 HEADER 的编码格式

HEADER 帧的格式

```

+-----+
| Pad Length? (8) |
+-----+
| E | Stream Dependency? (31) |
+-----+
| Weight? (8) |
+-----+
| Header Block Fragment (*) ...
+-----+
| Padding (*) ...
+-----+

```

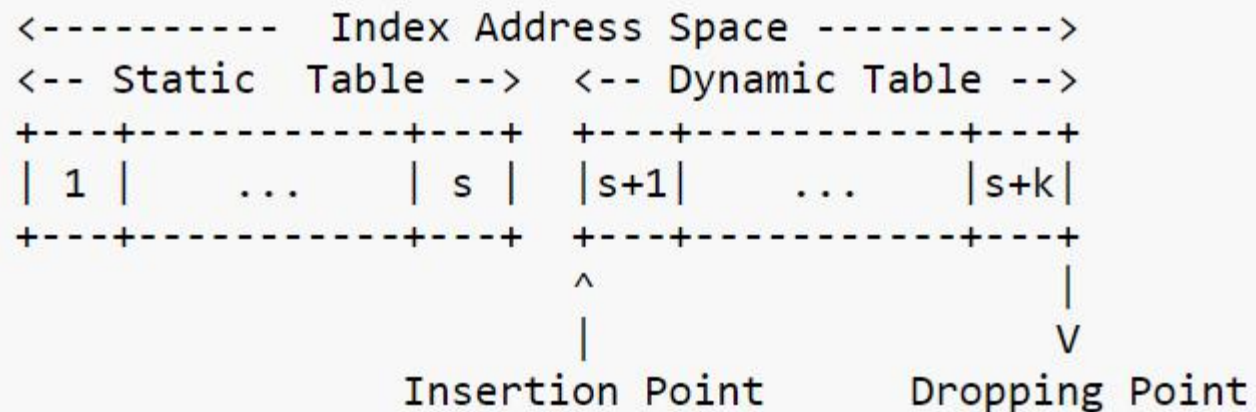
CONTINUATION 持续帧(type=0x9)

- 跟在 HEADER 帧或者 PUSH_PROMISE 帧之后, 补充完整的 HTTP 头部

```
+-----+
|           Header Block Fragment (*)           ...
+-----+
```

同一地址空间下的静态表与动态表

- 静态表：61 项
- 动态表：先入先出的淘汰策略
 - 动态表大小由 SETTINGS_HEADER_TABLE_SIZE 设置帧定义
 - 允许重复项
 - 初始为空



字面编码

- 组成
 - header name 和 value 以索引方式编码
 - header name 以索引方式编码，而 header value 以字面形式编码
 - header name 和 value 都以字面形式编码
- 可控制是否进入动态表
 - 进入动态表，供后续传输优化使用
 - 不进入动态表
 - 不进入动态表，并约定该头部永远不进入动态表

HEADER 二进制编码格式 (1)

- 名称与值都在索引表中 (包括静态表与动态表)
 - 编码方式: 首位传 1, 其余 7 位传索引号

0	1	2	3	4	5	6	7
+---+---+---+---+---+---+---+							
1	Index (7+)						
+---+---+---+---+---+---+---+							

- 例如
 - method: GET 在静态索引表中序号为 2, 其表示应为 1000 0010, HEX 表示为 82

HEADER 二进制编码格式 (2)

- 名称在索引表中, 值需要编码传递, 同时新增至动态表中
 - 前 2 位传 01
 - 名称举例
 - if-none-match 在静态索引表中序号为 41, 表示为 01101001, HEX 表示为 69
 - 值举例
 - "5cb816f5-19d8"
 - value长度 15 个字节, 采用 Huffman 编码(H 为1)
 - 10001100
 - 8c fe 5b 24 6f 05 c9 5b 58 2f c8 f7 f3

0	1	2	3	4	5	6	7
+	+	+	+	+	+	+	+
0	1	Index (6+)					
+	+	+	+	+	+	+	+
H		Value Length (7+)					
+	+	+	+	+	+	+	+
	Value String (Length octets)						
+	+	+	+	+	+	+	+

HEADER 二进制编码格式 (4)

- 名称在索引表中，值需要编码传递，且不更新至动态表中
 - 前 4 位传 0000

0	1	2	3	4	5	6	7
+	+	+	+	+	+	+	+
0	0	0	0	Index (4+)			
+	+	+	+	+	+	+	+
H	Value Length (7+)						
+	+	+	+	+	+	+	+
	Value String (Length octets)						
+	+	+	+	+	+	+	+

HEADER 二进制编码格式 (5)

- 名称、值都需要编码传递，且不更新至动态表中
 - 前 4 位传 0000

0	1	2	3	4	5	6	7
+	+	+	+	+	+	+	+
0	0	0	0			0	
+	+	+	+	+	+	+	+
H	Name Length (7+)						
+	+	+	+	+	+	+	+
	Name String (Length octets)						
+	+	+	+	+	+	+	+
H	Value Length (7+)						
+	+	+	+	+	+	+	+
	Value String (Length octets)						
+	+	+	+	+	+	+	+

动态表大小的两种控制方式

- 在 HEADER 帧中直接修改

0	1	2	3	4	5	6	7
+	-	-	+	-	-	+	-
	0		0		1		Max size (5+)
+	-	-	-	-	-	-	+

- 在 SETTING 帧中修改
 - SETTINGS_MAX_HEADER_LIST_SIZE (0x6)

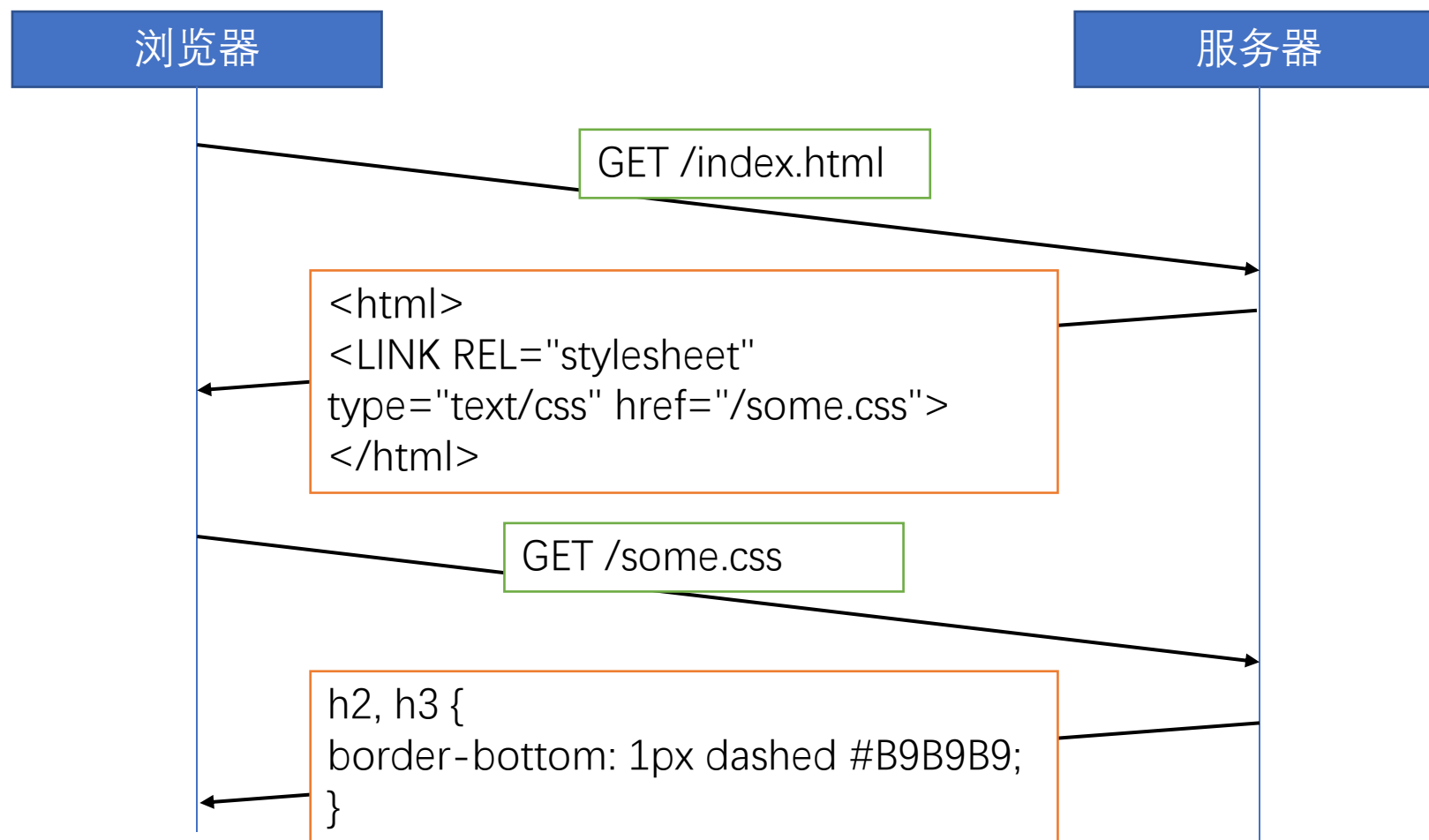
第 13 课 服务器端的主动消息推送

服务器推送的价值

- 提前将资源推送至浏览器缓存
- 特性
 - 推送可以基于已经发送的请求，例如客户端请求 html，主动推送 js 文件
- 实现方式
 - 推送资源必须对应一个请求
 - 请求由服务器端 PUSH_PROMISE 帧发送
 - 响应在偶数 ID 的 STREAM 中发送

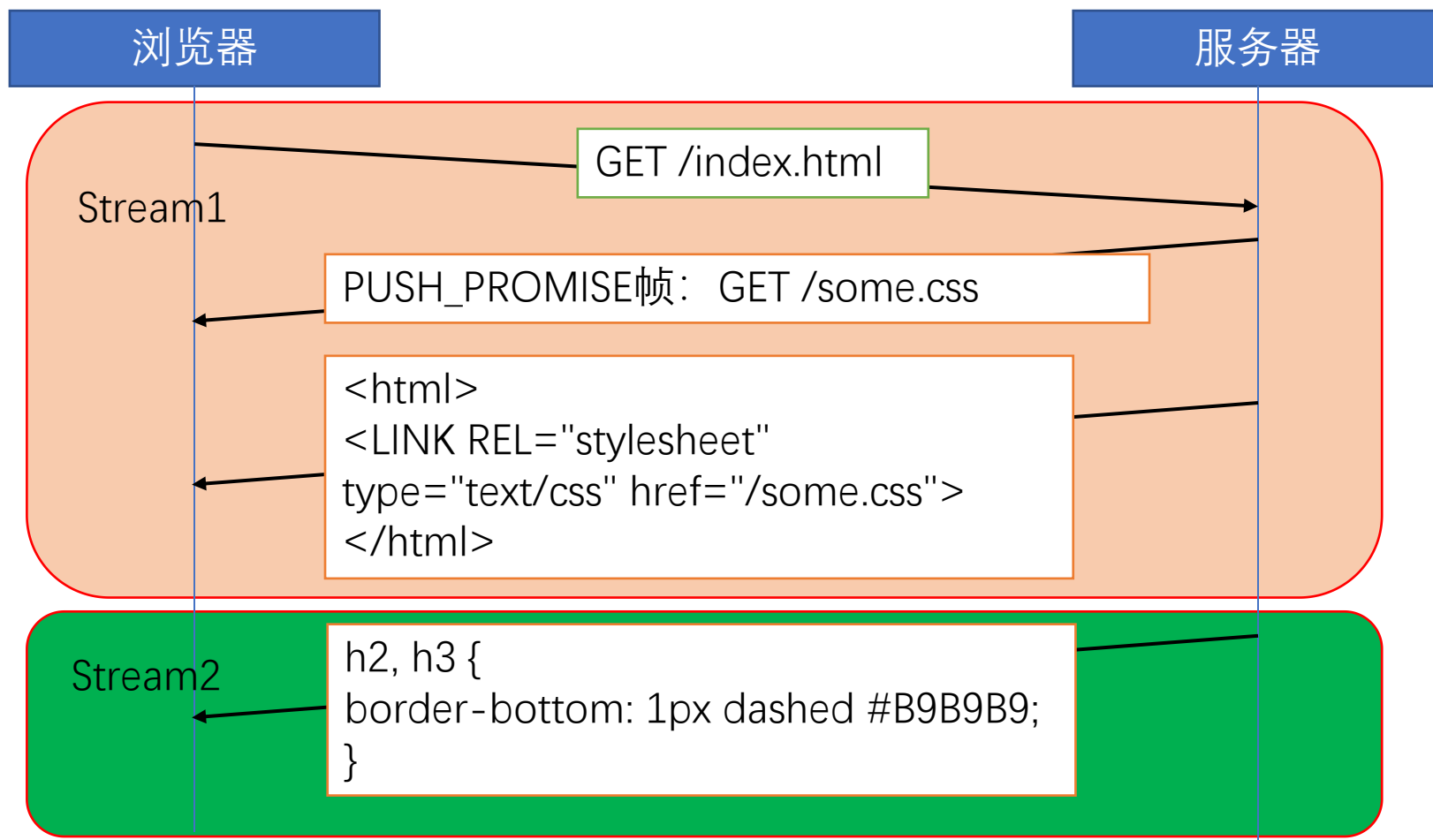
当获取 HTML 后，需要 CSS 资源时

- 浏览器触发方式：需要两次往返！

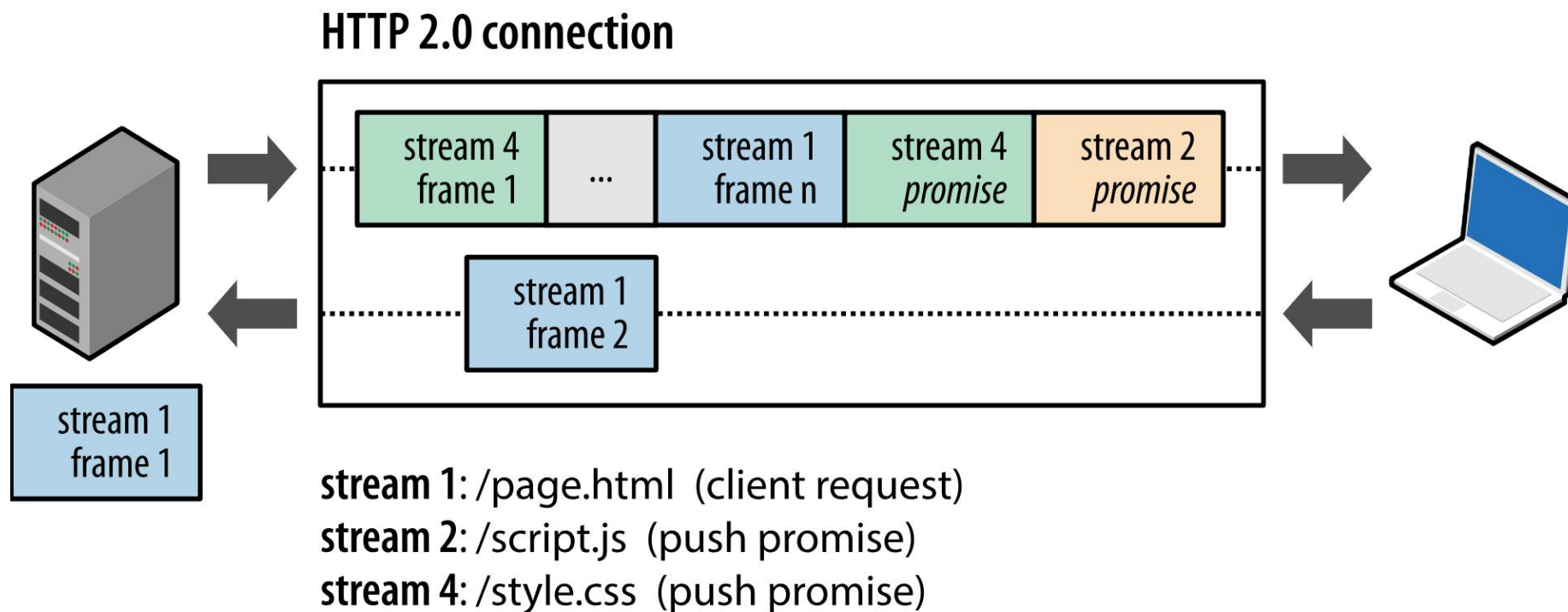


• PUSH_PROMISE 方式

- 在 Stream1 中通知客户端 CSS 资源即将来临
- 在 Stream2 中发送 CSS 资源 (Stream1 和 2 可以并发!)



服务器推送 PUSH



PUSH 帧的格式

- PUSH_PROMISE 帧, type=0x5, 只能由服务器发送

```
+-----+
|Pad Length? (8)|
+-----+-----+
|R|                Promised Stream ID (31)                |
+-----+-----+
|                Header Block Fragment (*)                ...
+-----+-----+
|                Padding (*)                                ...
+-----+-----+
```

PUSH 推送模式的禁用

- SETTINGS_ENABLE_PUSH (0x2)
 - 1表示启用推送功能
 - 0表示禁用推送功能

第 14 课 stream 的状态变迁

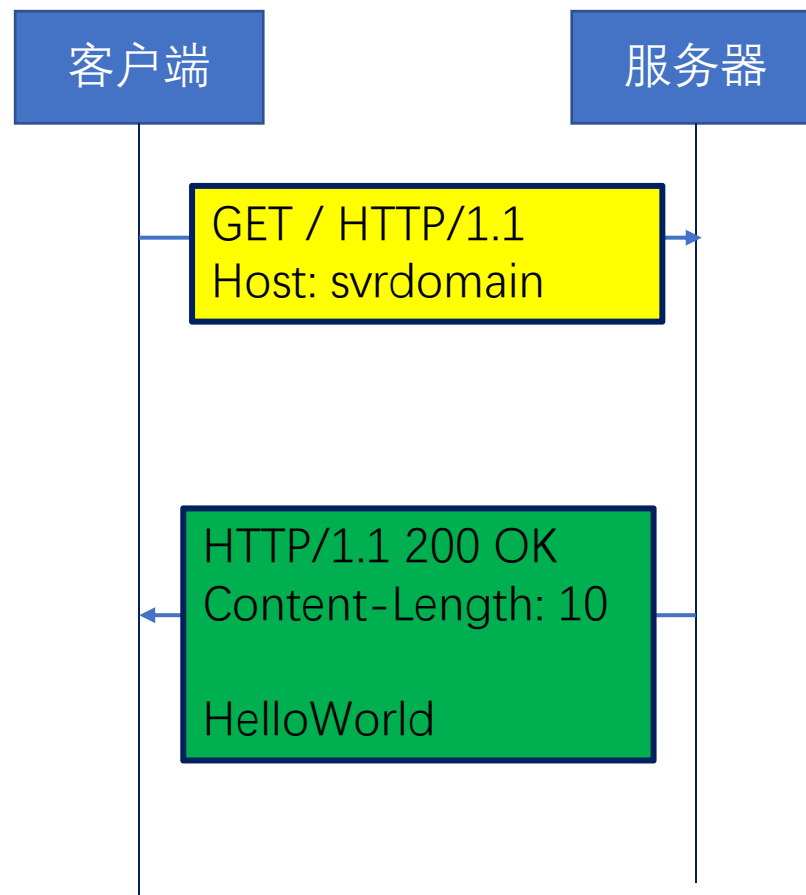
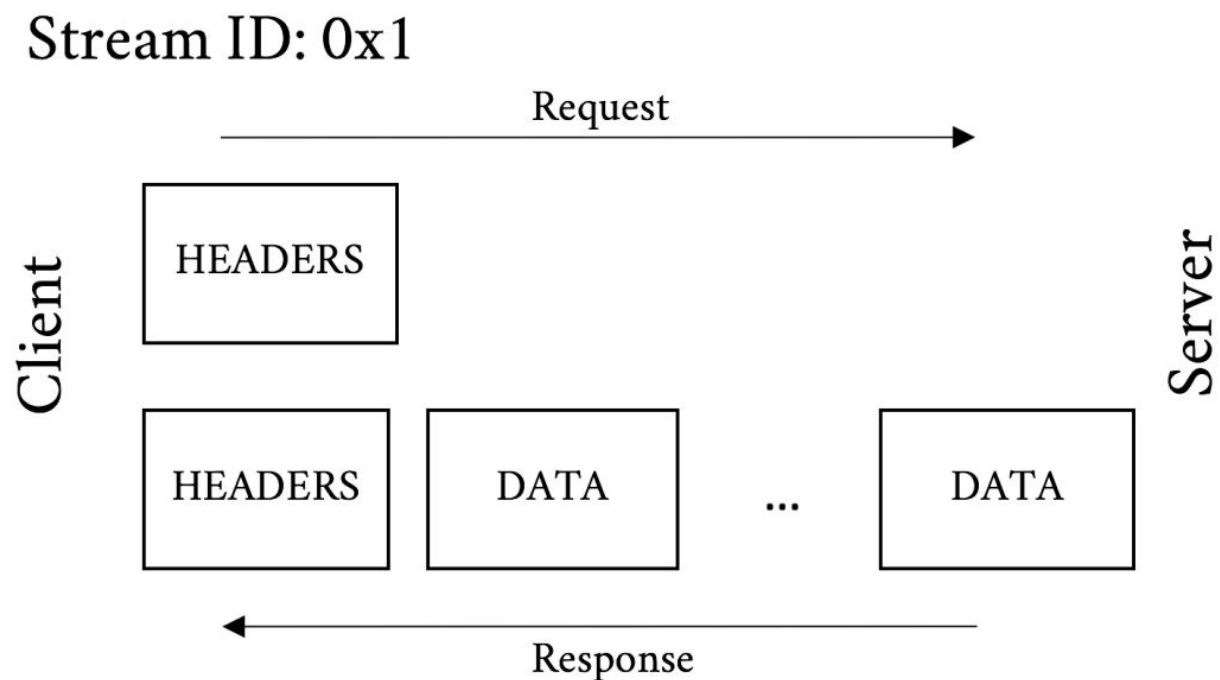
Stream 特性

- 一条 TCP 连接上，可以并发存在多个处于 OPEN 状态的 Stream
- 客户端或者服务器都可以创建新的 Stream
- 客户端或者服务器都可以首先关闭 Stream
- 同一条 Stream 内的 Frame 帧是有序的
- 从 Stream ID 的值可以轻易分辨 PUSH 消息
 - 所有为发送 HEADER/DATA 消息而创建的流，从 1、3、5 等递增奇数开始
 - 所有为发送 PUSH 消息而创建的流，从 2、4、6 等递增偶数开始

Message 特性

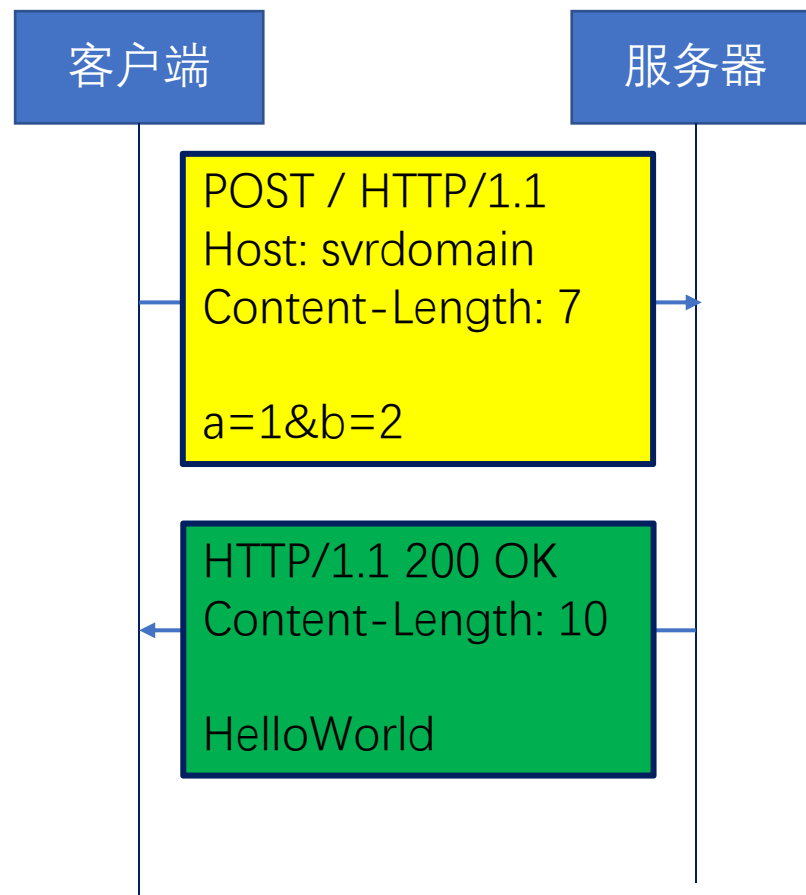
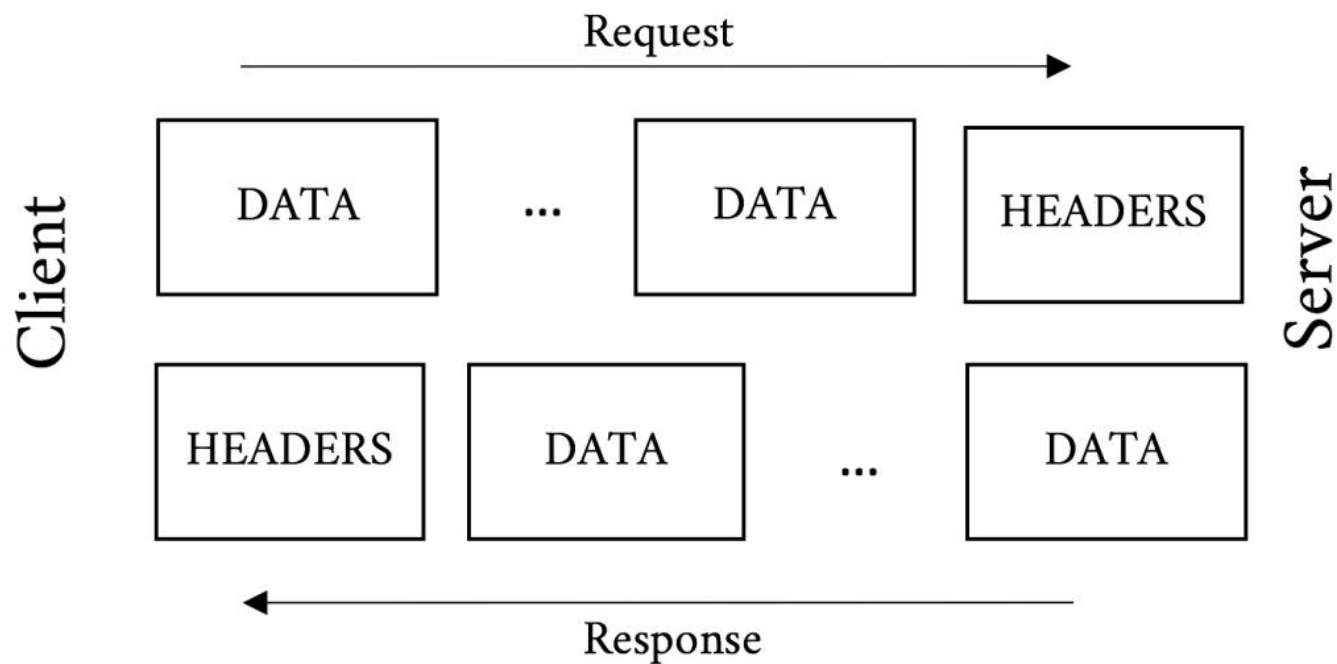
- 一条 HTTP Message 由 1 个 HEADER (可能含有 0 个或者多个持续帧构成) 及 0 个或者多个 DATA 帧构成
- HEADER 消息同时包含 HTTP/1.1 中的 start line 与 headers 部分
- 取消 HTTP/1.1 中的不定长 Chunk 消息

GET 消息发送示例



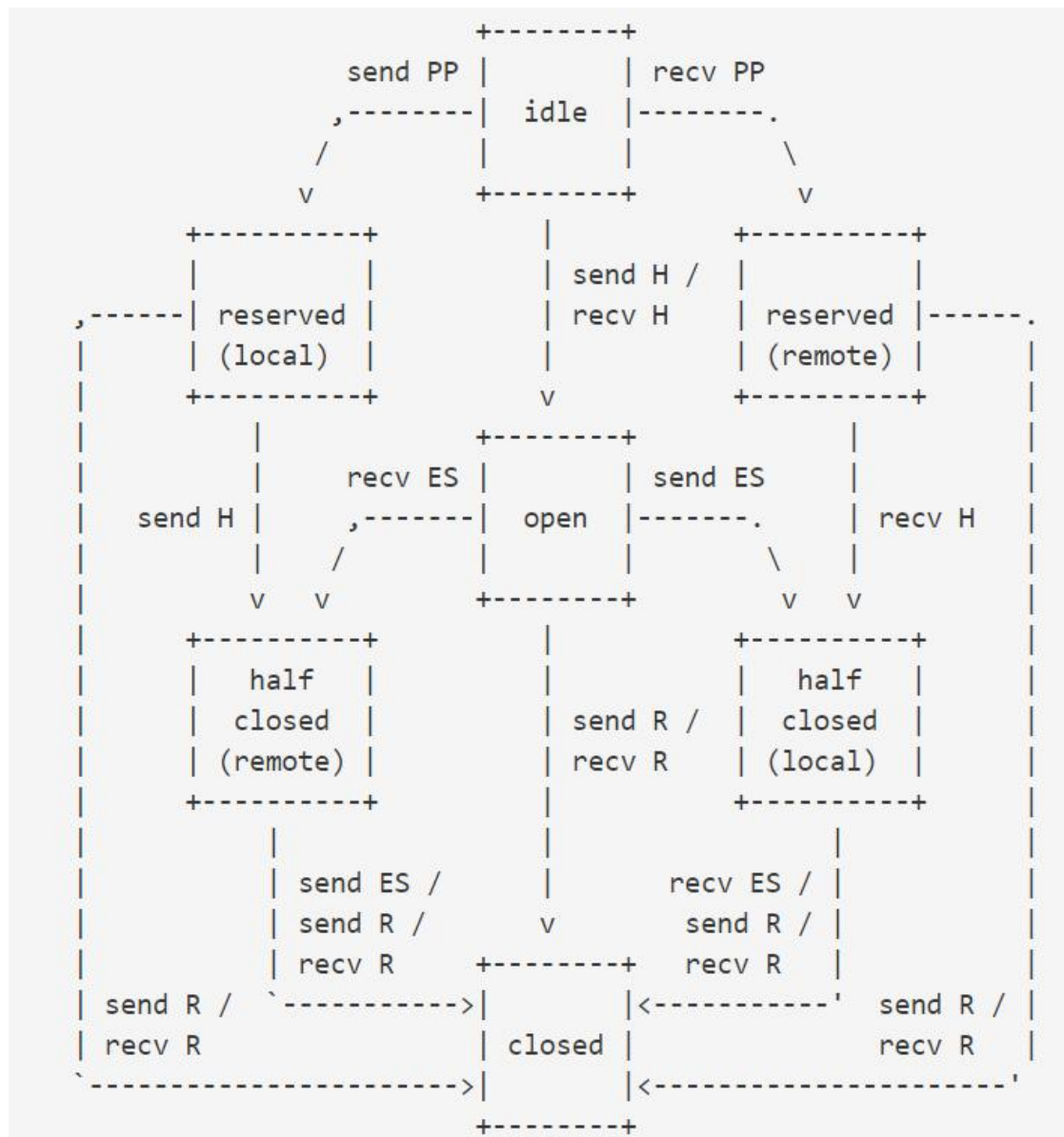
POST 消息发送示例

Stream ID: 0x1



Stream 流的状态

- 帧符号
 - H: HEADERS 帧
 - PP: PUSH_PROMISE 帧
 - ES: END_STREAM 标志位
 - R: RST_STREAM 帧
- 流状态
 - idle: 起始状态
 - closed
 - open: 可以发送任何帧
 - half closed 单向关闭
 - remote: 不再接收数据帧
 - local: 不能再发送数据帧
 - reserved
 - remote
 - local



第 15 课 RST_STREAM 帧及常见错误码

RST_STREAM 帧 (type=0x3)

- HTTP2 多个流共享同一连接, RST 帧允许立刻终止一个未完成的流
- RST_STREAM 帧不使用任何 flag
- RST_STREAM 帧的格式



常见错误码 (1)

- NO_ERROR (0x0): 没有错误。GOAWAY帧优雅关闭连接时可以使用此错误码
- PROTOCOL_ERROR (0x1): 检测到不识别的协议字段
- INTERNAL_ERROR (0x2): 内部错误
- FLOW_CONTROL_ERROR (0x3): 检测到对端没有遵守流控策略
- SETTINGS_TIMEOUT (0x4): 某些设置帧发出后需要接收端应答, 在期待时间内没有得到应答则由此错误码表示

常见错误码 (2)

- STREAM_CLOSED (0x5): 当Stream已经处于半关闭状态不再接收Frame帧时, 又接收到了新的Frame帧
- FRAME_SIZE_ERROR (0x6): 接收到的Frame Size不合法
- REFUSED_STREAM (0x7): 拒绝先前的Stream流的执行
- CANCEL (0x8): 表示Stream不再存在
- COMPRESSION_ERROR (0x9): 对HPACK压缩算法执行失败

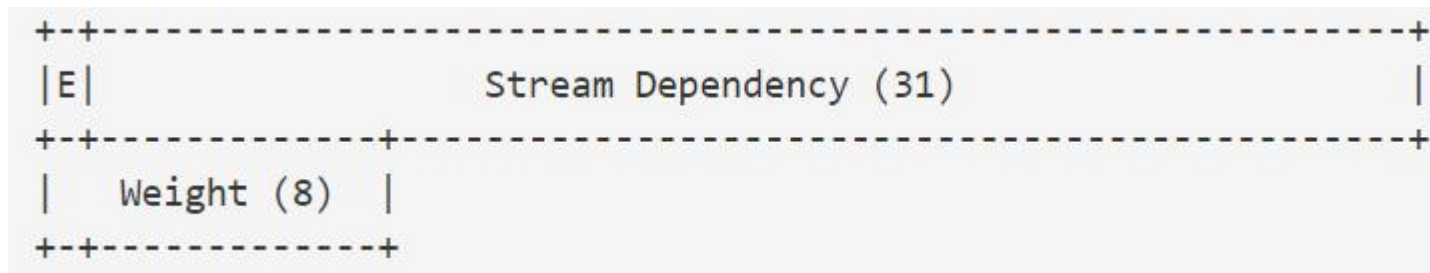
常见错误码 (3)

- CONNECT_ERROR (0xa): 连接失败
- ENHANCE_YOUR_CALM (0xb): 检测到对端的行为可能导致负载的持续增加, 提醒对方 “冷静” 一点
- INADEQUATE_SECURITY (0xc): 安全等级不够
- HTTP_1_1_REQUIRED (0xd): 对端只能接受HTTP/1.1协议

第 16 课 不同请求的优先级

Priority 优先级设置帧

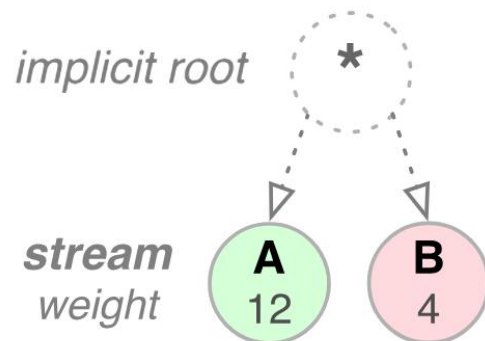
- 帧类型: type=0x2
- 不使用 flag 标志位字段
- Stream Dependency: 依赖流
- Weight权重: 取值范围为 1 到 256。默认权重16
- 仅针对 Stream 流, 若 ID 为 0 试图影响连接, 则接收端必须报错
- 在 idle 和 closed 状态下, 仍然可以发送 Priority 帧



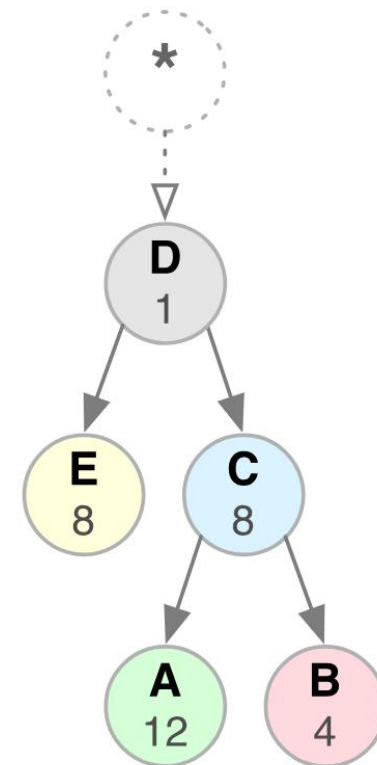
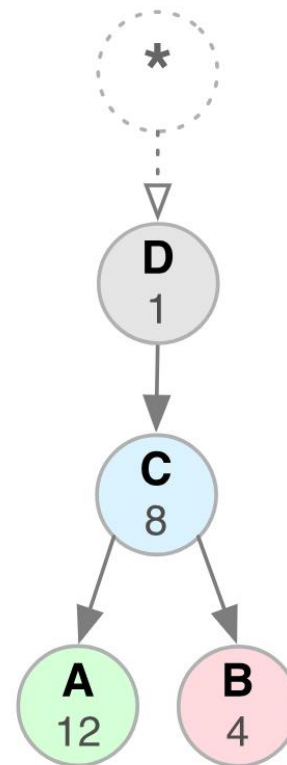
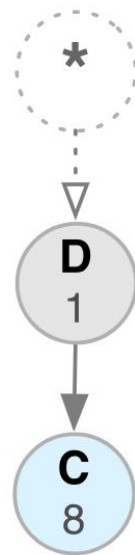
HEADER 帧的格式

```
+-----+
|Pad Length? (8)|
+-+-----+-----+
|E|                Stream Dependency? (31)                |
+-+-----+-----+
|  Weight? (8)  |
+-+-----+-----+
|                Header Block Fragment (*)                ...
+-----+-----+
|                Padding (*)                                ...
+-----+-----+
```

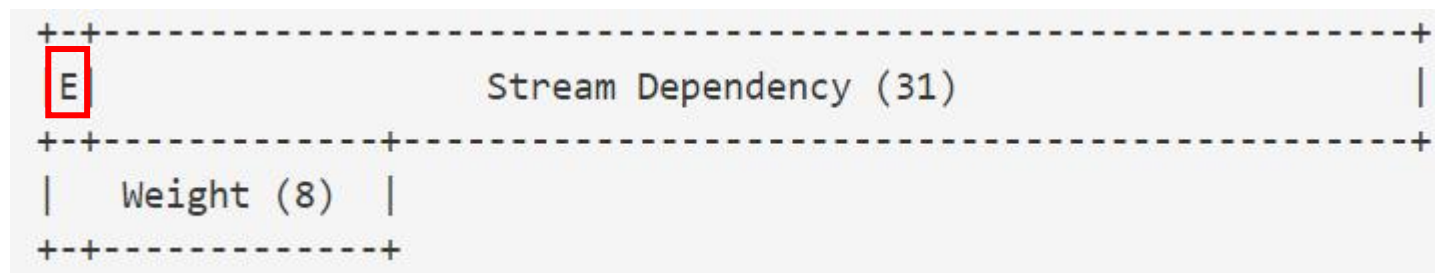
数据流优先级



- 每个数据流有优先级 (1-256)
- 数据流间可以有依赖关系



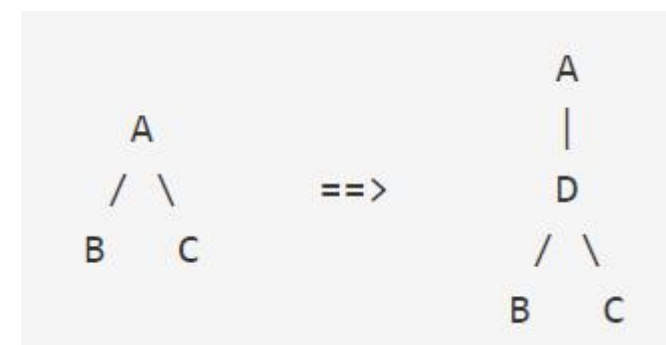
exclusive 标志位



• E 设为 0



E 设为 1

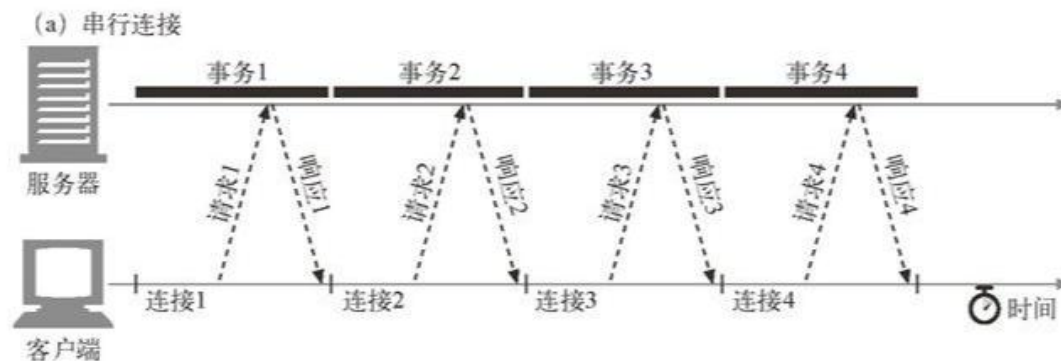


第 17 课 不同于 TCP 的流量控制

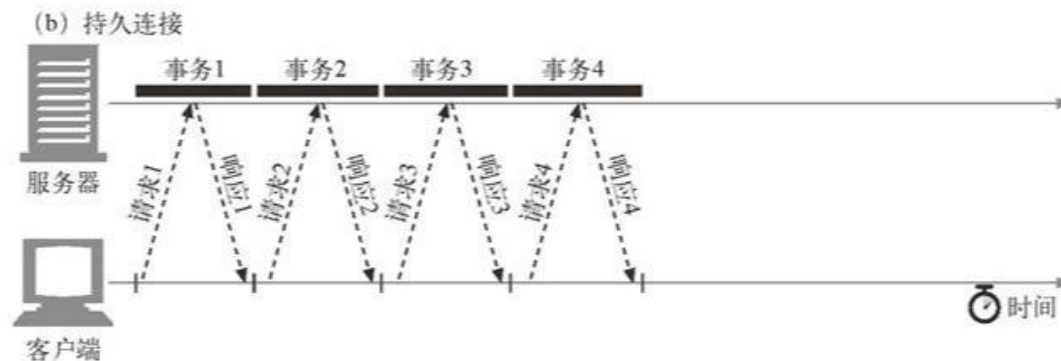
为什么需要 HTTP/2 应用层流控? (1)

- HTTP/1.1 中由 TCP 层进行流量控制
 - 前提: HTTP/1 的 TCP 连接上没有多路复用

- TCP接收窗口
- TCP拥塞控制

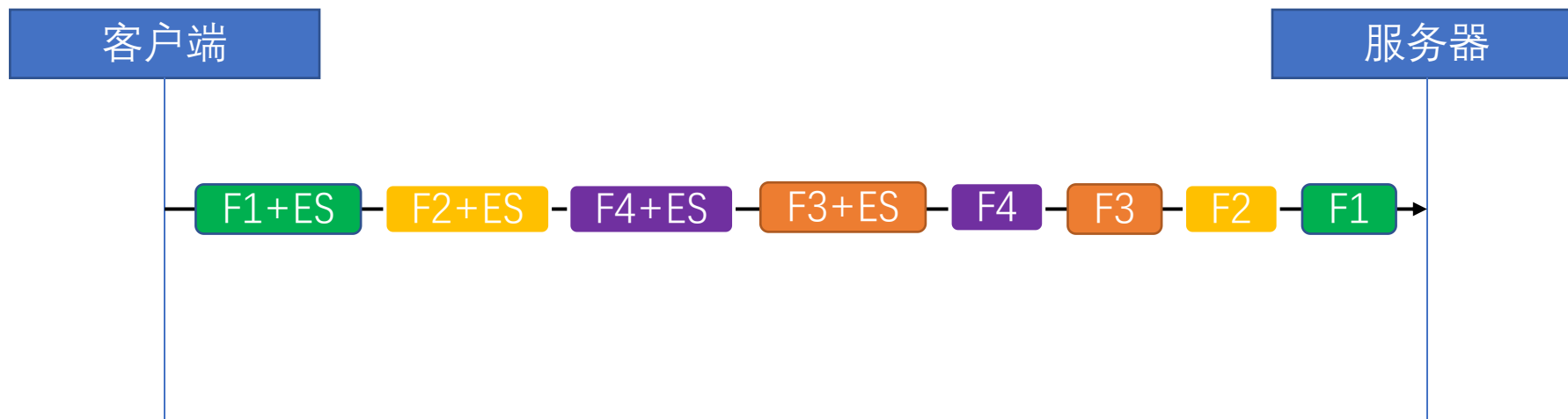


- TCP接收窗口



为什么需要HTTP/2 应用层流控? (2)

- HTTP/2 中，多路复用意味着多个 Stream 必须共享 TCP 层的流量控制
 - 问题：多 Stream 争夺 TCP 的流控制，互相干扰可能造成 Stream 阻塞



- 代理服务器内存有限，上下游网速不一致时，通过流控管理内存

由应用层决定发送速度

- HTTP/2 中的流控制既针对单个 Stream，也针对整个 TCP 连接
 - 客户端与服务器都具备流量控制能力
 - 单向流控制：发送和接收独立设定流量控制
 - 以信用为基础：接收端设定上限，发送端应当遵循接收端发出的指令
 - 流量控制窗口（流或者连接）的初始值是 65535 字节
 - 只有 DATA 帧服从流量控制
 - 流量控制不能被禁用

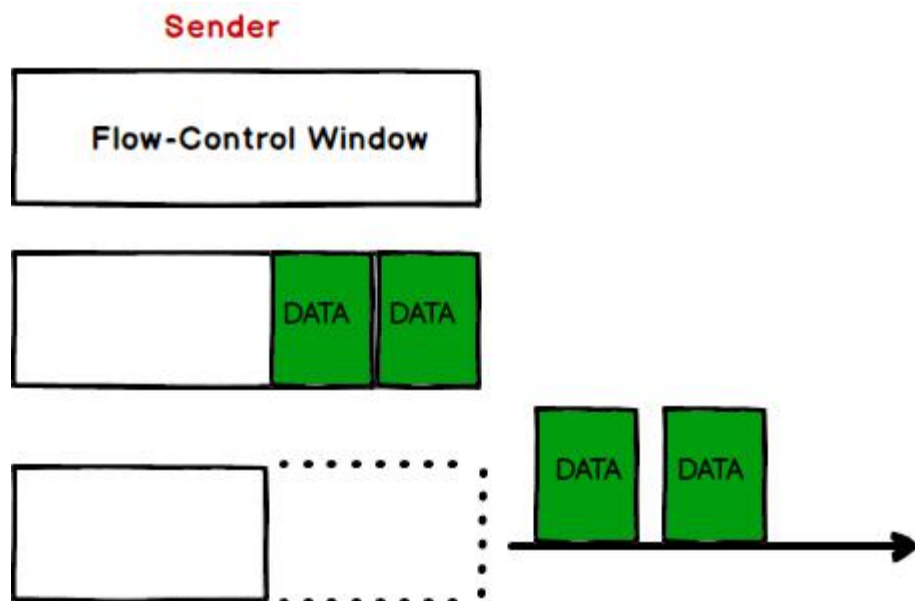
WINDOW_UPDATE 帧

- type=0x8, 不使用任何 flag
- 窗口范围 1 to $2^{31}-1$ (2,147,483,647)字节
 - 0 是错误的, 接收端应返回 `PROTOCOL_ERROR`
- 当 Stream ID 为 0 时表示对连接流控, 否则为对 Stream 流控
- 流控仅针对直接建立 TCP 连接的两端
 - 代理服务器并不需要透传 WINDOW_UPDATE 帧
 - 接收端的缩小流控窗口会最终传递到源发送端

```
+---+-----+
|R|           Window Size Increment (31)           |
+---+-----+
```

流控制窗口

- 窗口大小由接收端告知
- 窗口随着 DATA 帧的发送而减少

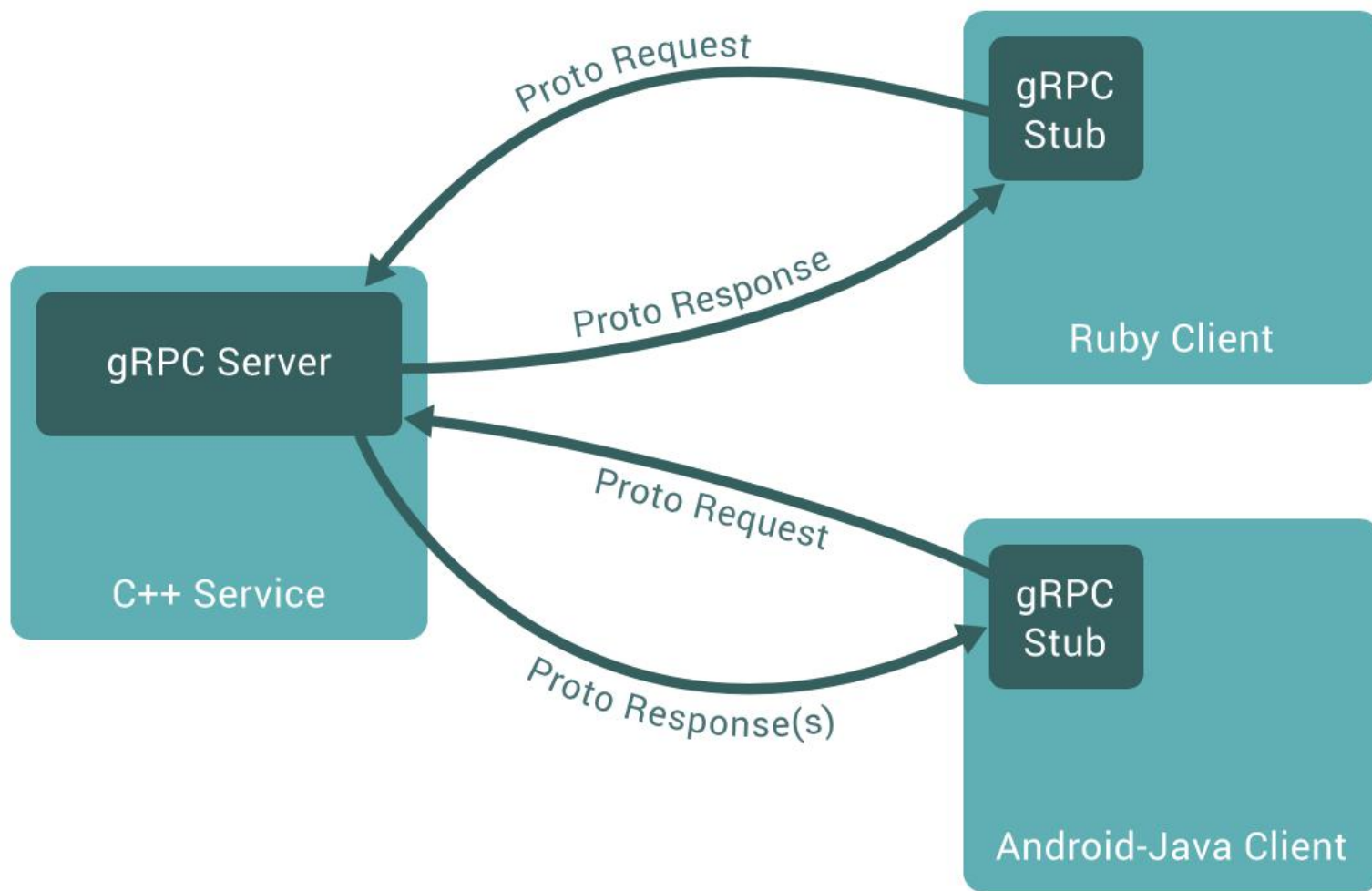


SETTINGS_MAX_CONCURRENT_STREAMS 并发流

- 并发仅统计 open 或者 half-close 状态的流（不包含用于推送的 reserved 状态）
- 超出限制后的错误码
 - PROTOCOL_ERROR
 - REFUSED_STREAM

第 18 课 HTTP/2 与 gRPC 框架

gRPC: 支持多语言编程、基于 HTTP/2 通讯的中间件



gRPC 测试

- 官网: <https://grpc.io/>
- 基于 Python 语言搭建测试环境
 - <https://grpc.io/docs/quickstart/python/>
 - 测试程序
 - git clone -b v1.21.0 <https://github.com/grpc/grpc>
 - cd grpc/examples/python/helloworld
 - 服务器: python greeter_server.py
 - 客户端: python greeter_client.py
 - 注意
 - 欲抓取环回报文, 请安装时勾选【install Npcap in Winpcap API-Compatible Mode】
 - 如果 Npcap Loopback Adapter 未抓取到环回报文, 请尝试其他接口
 - 若 50051 端口未被识别为 http/2, 请手动设置“解码为 HTTP/2”

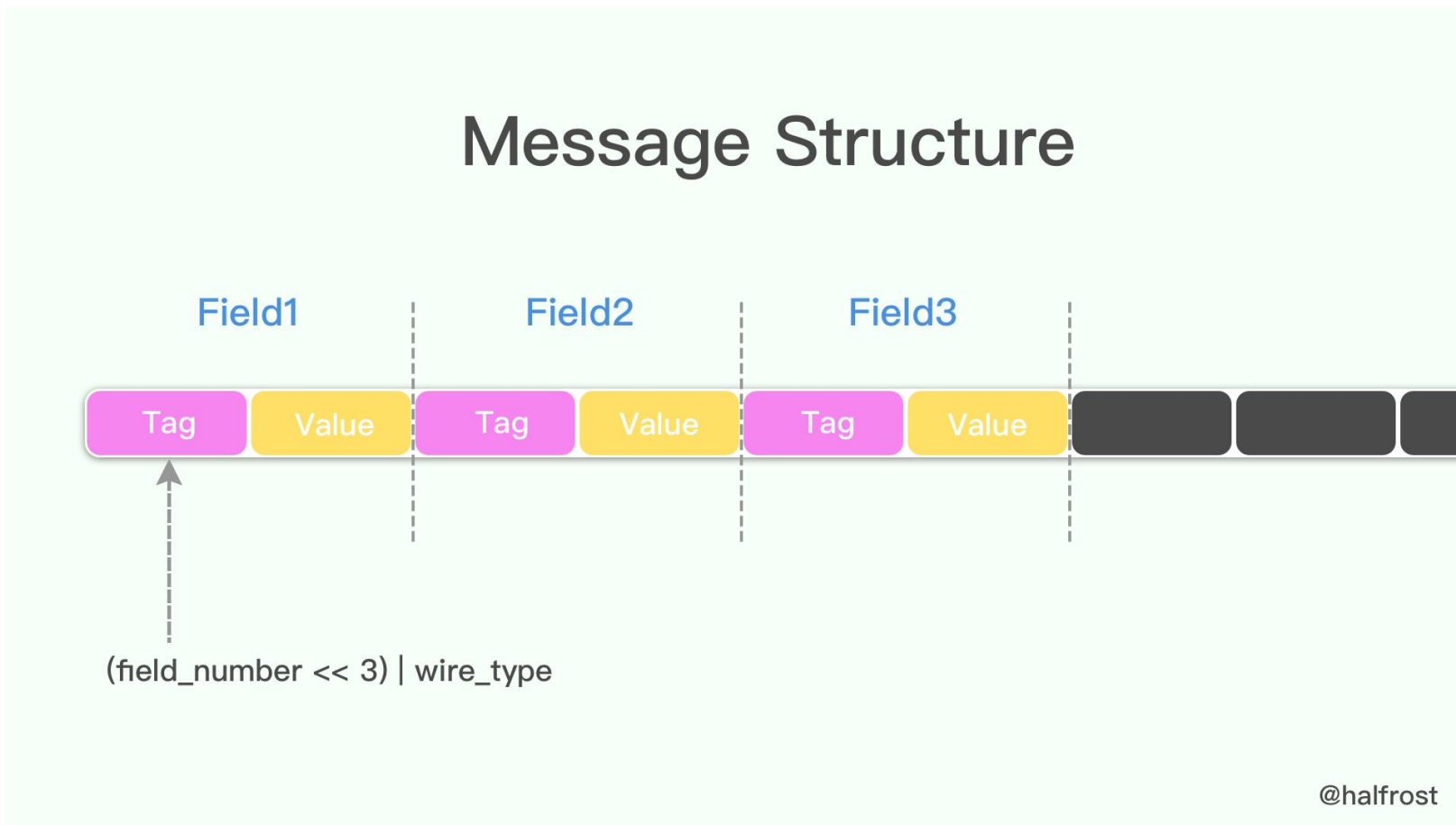
helloworld.proto

```
// The greeting service definition.
service Greeter {
    // Sends a greeting
    rpc SayHello (HelloRequest) returns (HelloReply) {}
}

// The request message containing the user's name.
message HelloRequest {
    string name = 1;
}

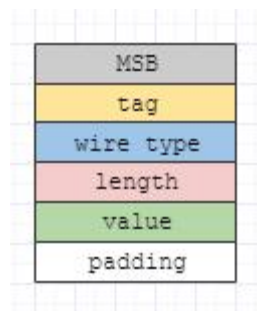
// The response message containing the greetings
message HelloReply {
    string message = 1;
}
```

Protocol Buffers 编码：消息结构



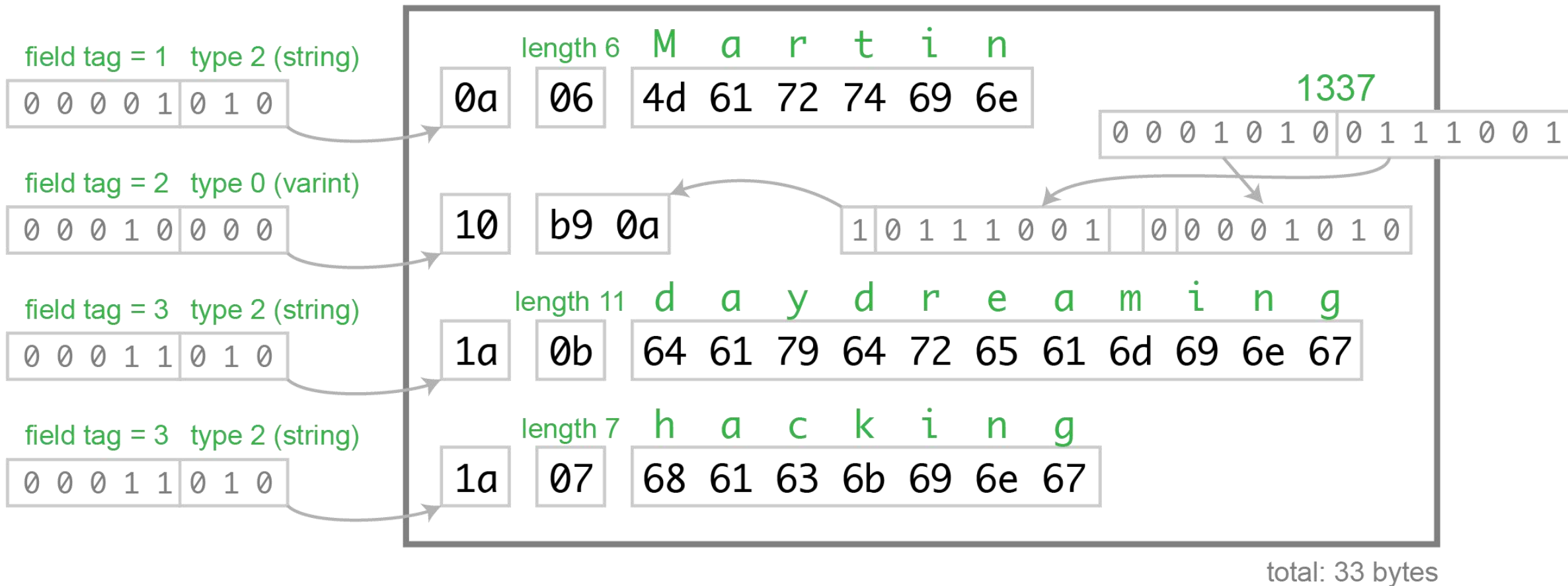
Protocol Buffers 编码：数据类型 Wire Type

Type	Meaning	Used For
0	Varint	int32, int64, uint32, uint64, sint32, sint64, bool, enum
1	64-bit	fixed64, sfixed64, double
2	Length-delimited	string, bytes, embedded messages, packed repeated fields
3	Start group	groups (deprecated)
4	End group	groups (deprecated)
5	32-bit	fixed32, sfixed32, float



Protocol Buffers 字符串编码举例

Protocol Buffers



第 19 课 HTTP/2 的问题及 HTTP/3 的意义

TCP 以及 TCP+TLS 建链握手过多的问题

TCP

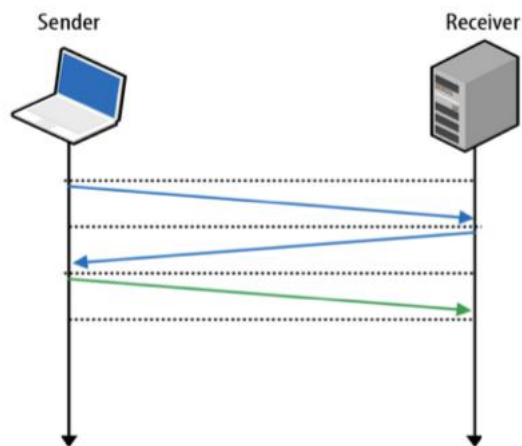


Figure 2-1. Three-way handshake

TCP + TLS

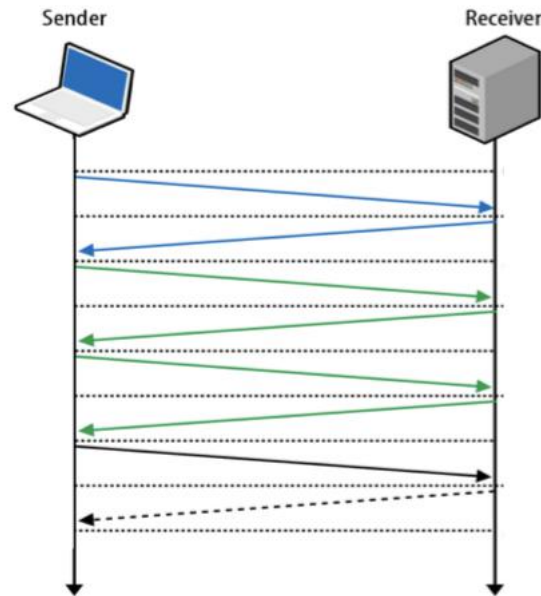
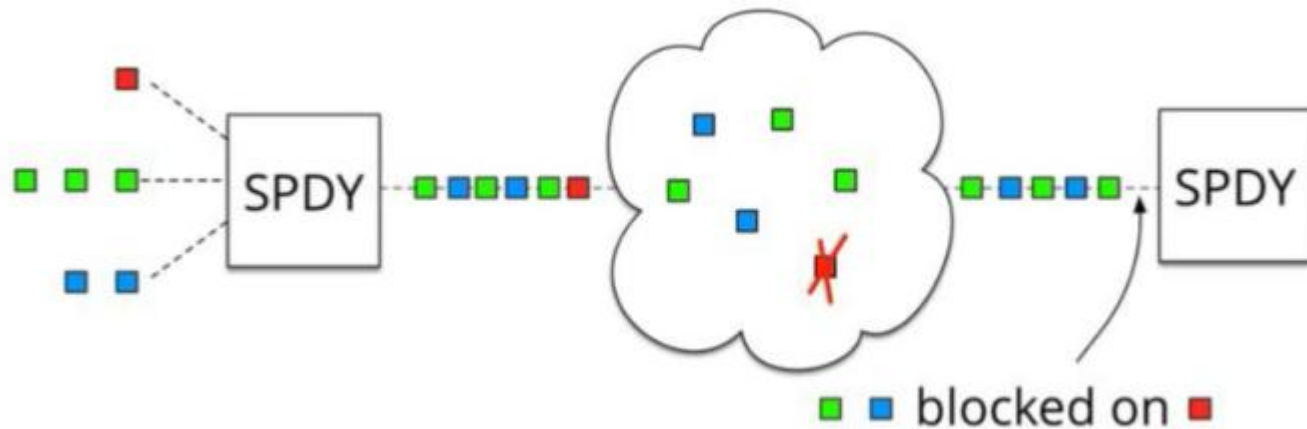
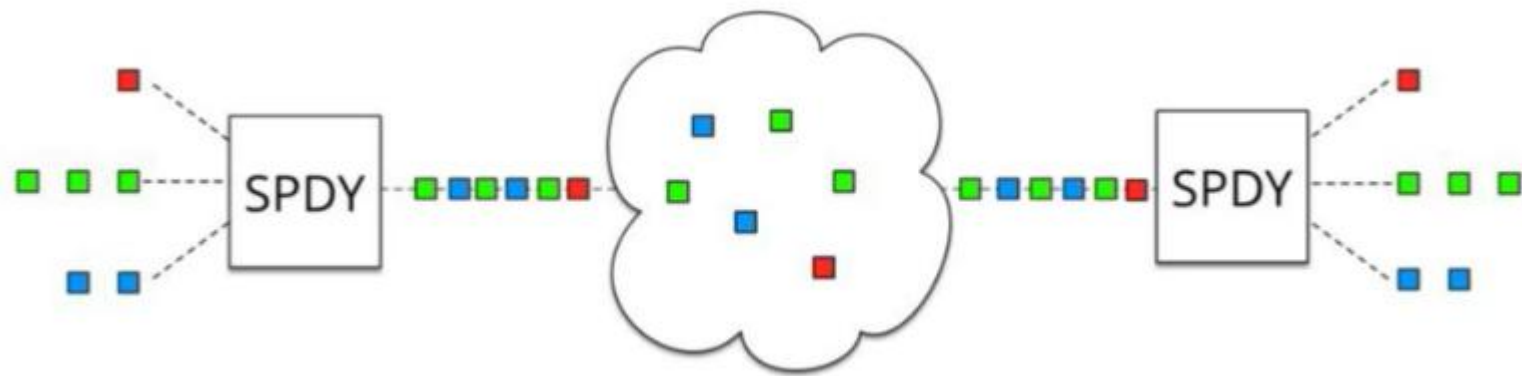


Figure 4-2. TLS handshake protocol

多路复用与 TCP 的队头阻塞问题

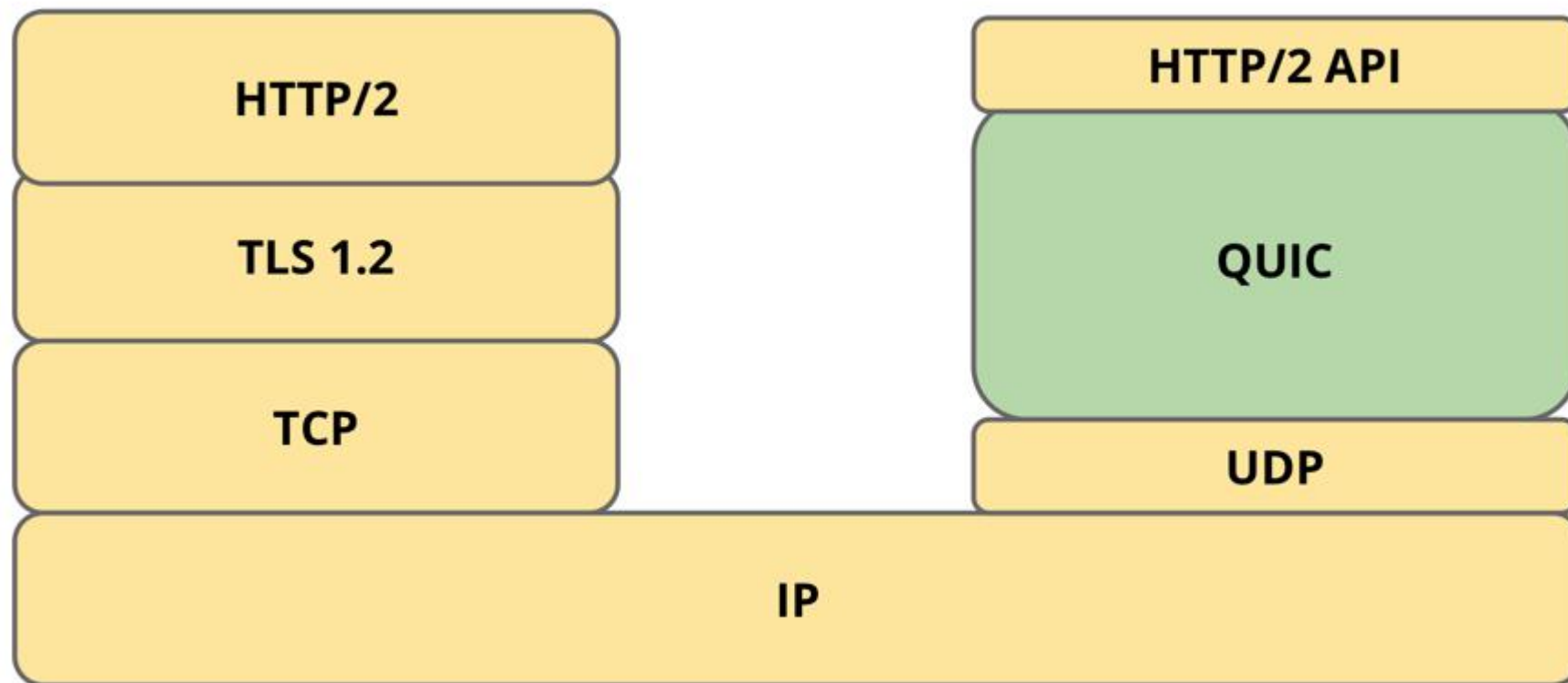
- 资源的有序到达



TCP的问题

- 由操作系统内核实现，更新缓慢

QUIC 协议在哪一层?



使 Chrome 支持 QUIC

- <chrome://flags/#enable-quic>

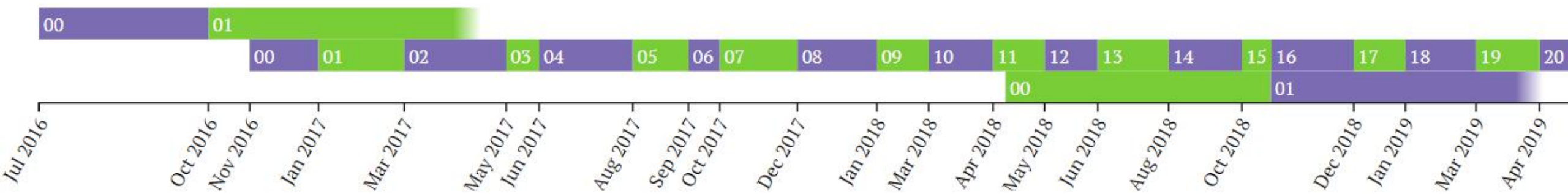
- Experimental QUIC protocol

Enable experimental QUIC protocol support. – Mac, Windows, Linux, Chrome OS, Android
[#enable-quic](#)

Enabled ▼

IETF QUIC 协议草案

- IETF draft 20: <https://tools.ietf.org/html/draft-ietf-quic-http-20>
- <https://datatracker.ietf.org/doc/draft-ietf-quic-transport/>

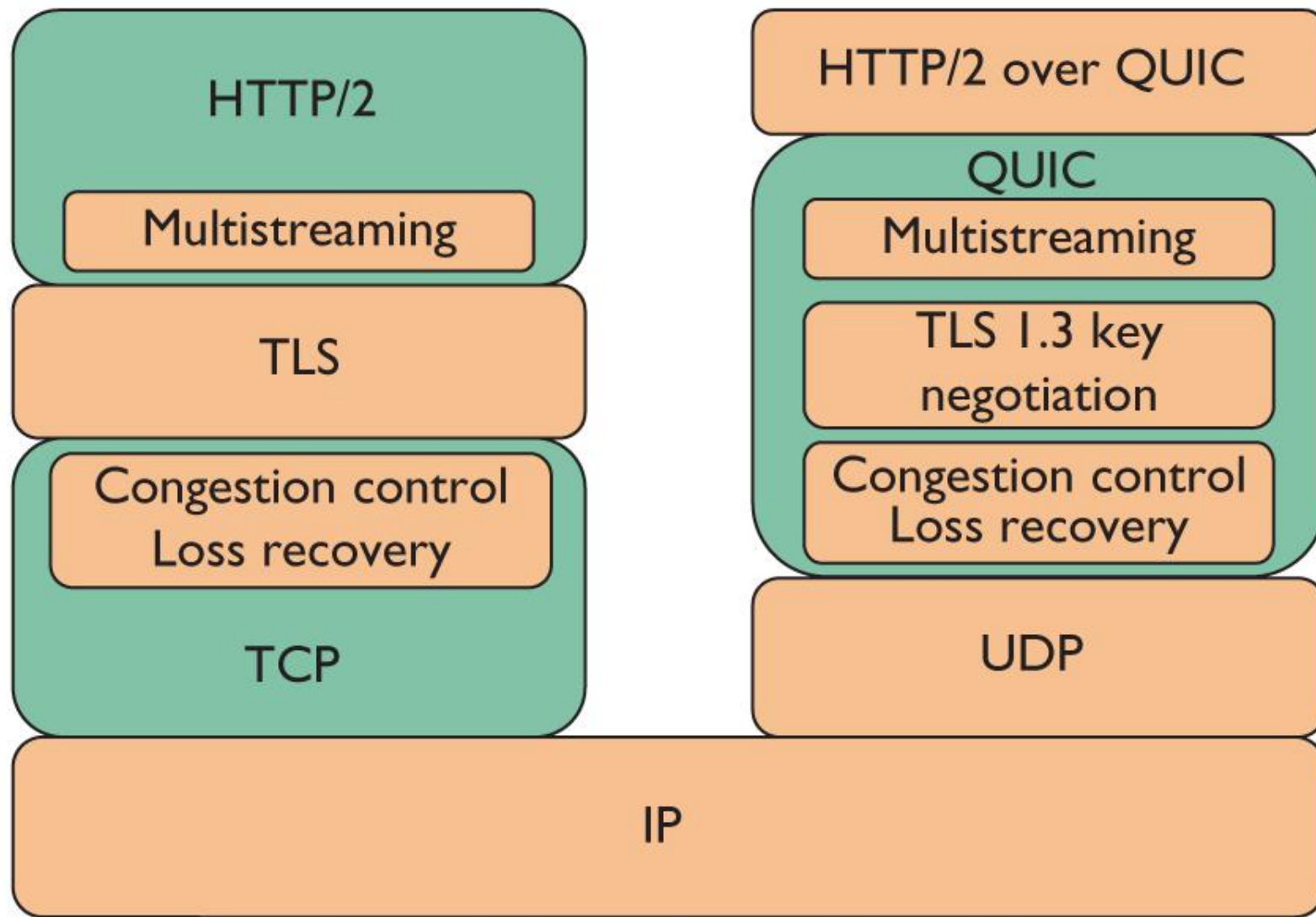


QUIC 协议组的 Milestones

Date	✦ Milestone
May 2020	Multipath extension document to IESG
Dec 2019	Working group adoption of Multipath extension document
Jul 2019	QPACK: Header Compression for HTTP over QUIC to IESG
Jul 2019	Header Compression for HTTP over QUIC to IESG
Jul 2019	Version-Independent Properties of QUIC to IESG
Jul 2019	QUIC Applicability and Manageability Statement to IESG
Jul 2019	HTTP/2 mapping document to IESG
Jul 2019	TLS 1.3 Mapping document to IESG
Jul 2019	Loss detection and Congestion Control document to IESG
Jul 2019	Core Protocol document to IESG
Done	Working group adoption of Version-Independent Properties of QUIC
Done	Working group adoption of Header Compression for HTTP over QUIC
Done	Working group adoption of QUIC Applicability and Manageability Statement
Done	Working group adoption of HTTP/2 mapping document
Done	Working group adoption of TLS 1.3 mapping document
Done	Working group adoption of Loss detection and Congestion Control document
Done	Working group adoption of Core Protocol document

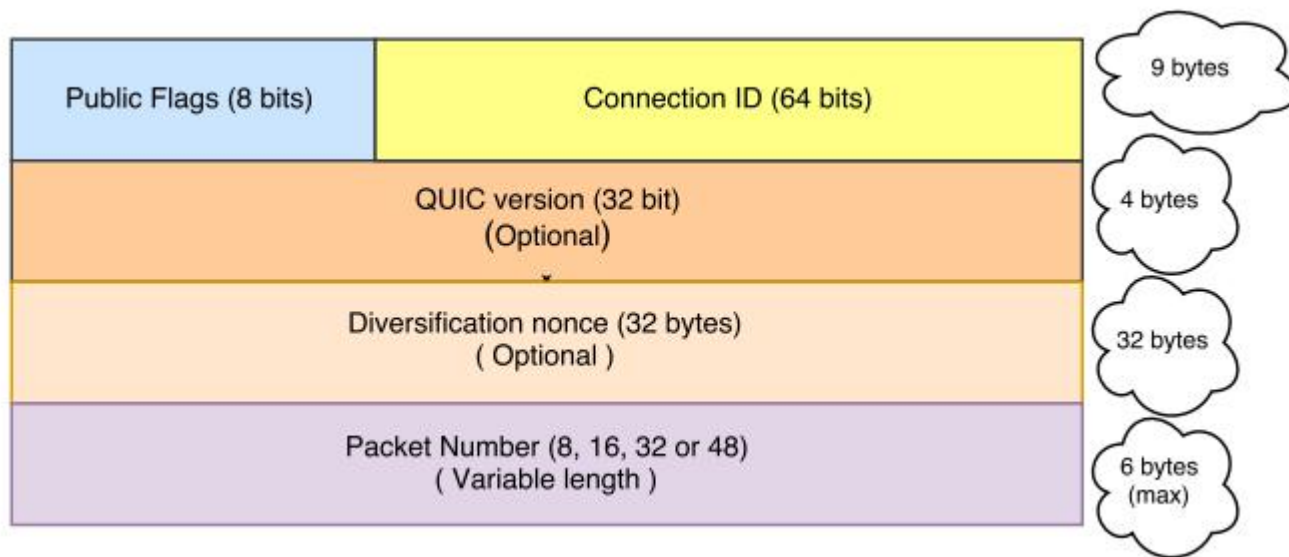
第 20 课 HTTP/3: QUIC 协议概述

HTTP/3 与 QUIC 协议



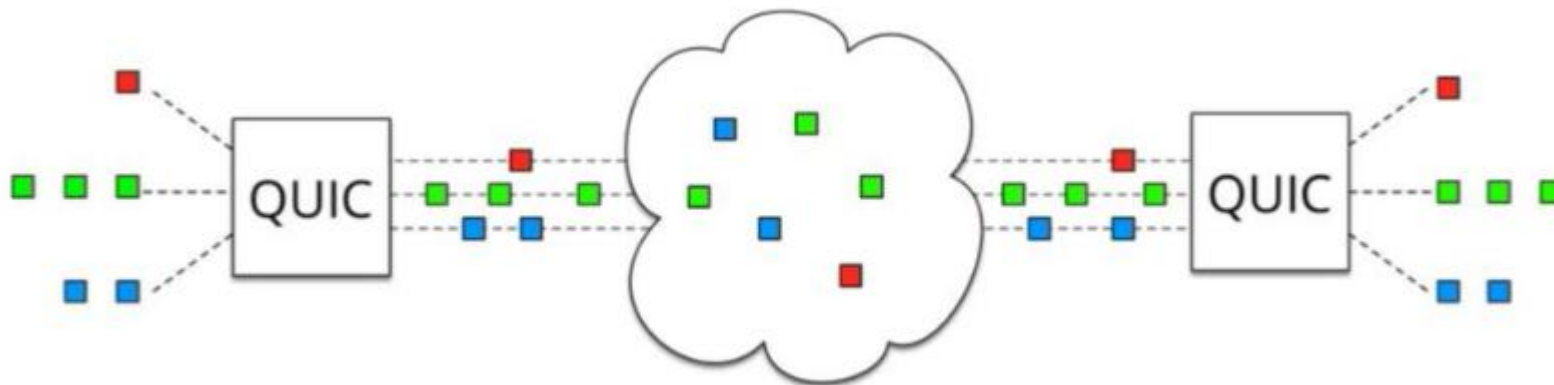
HTTP3 的连接迁移

- 允许客户端更换 IP 地址、端口后，仍然可以复用前连接



解决了队头阻塞问题的 HTTP3

- UDP 报文：先天没有队列概念



HTTP3: 1RTT 完全握手

Client

Server

Initial[0]: CRYPTO[CH] ->

Initial[0]: CRYPTO[SH] ACK[0]
Handshake[0]: CRYPTO[EE, CERT, CV, FIN]
<- 1-RTT[0]: STREAM[1, "..."]

Initial[1]: ACK[0]

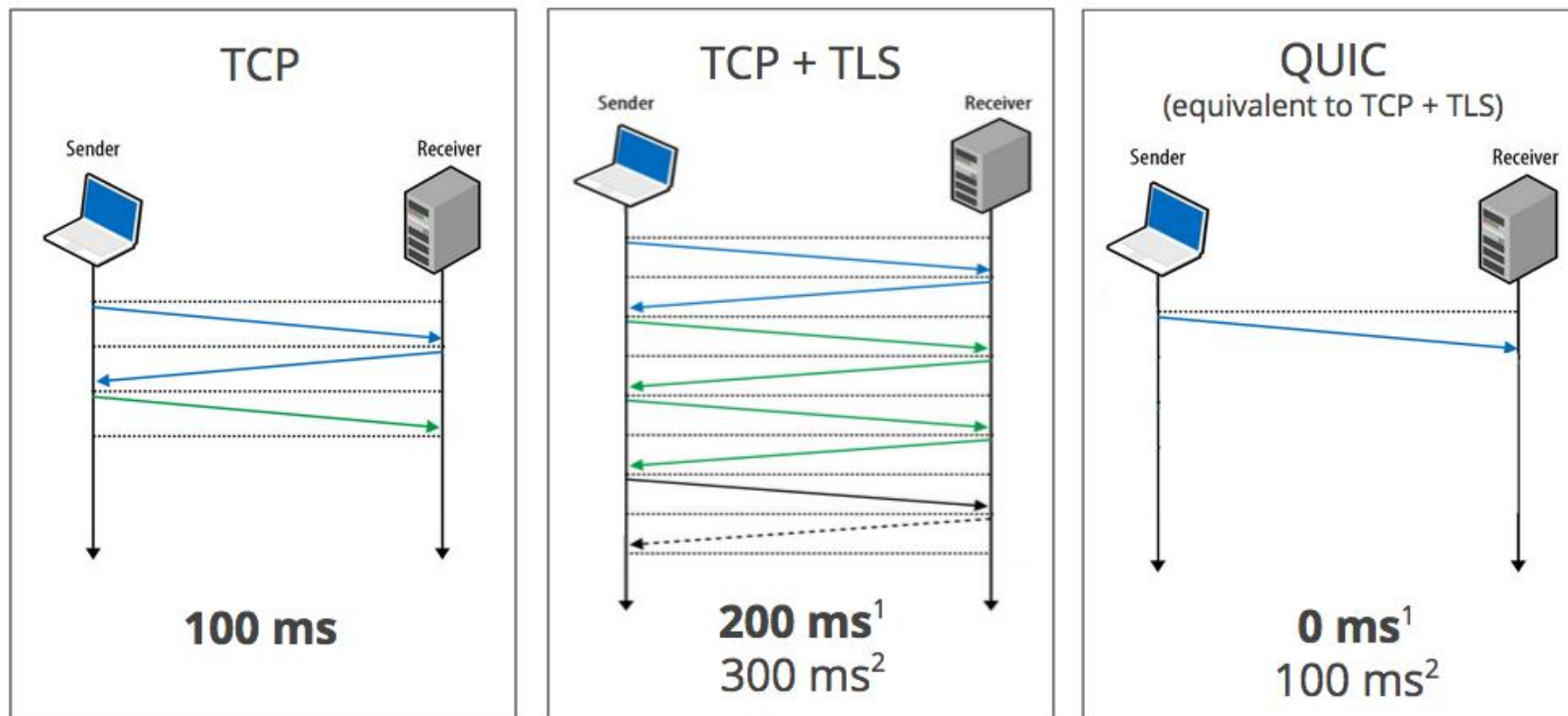
Handshake[0]: CRYPTO[FIN], ACK[0]

1-RTT[0]: STREAM[0, "..."], ACK[0] ->

1-RTT[1]: STREAM[3, "..."], ACK[0]
<- Handshake[1]: ACK[0]

会话恢复场景下的 0RTT 握手

Zero RTT Connection Establishment



1. Repeat connection
2. Never talked to server before

HTTP3: 0RTT 恢复会话握手

Client

Server

Initial[0]: CRYPTO[CH]
0-RTT[0]: STREAM[0, "..."] ->

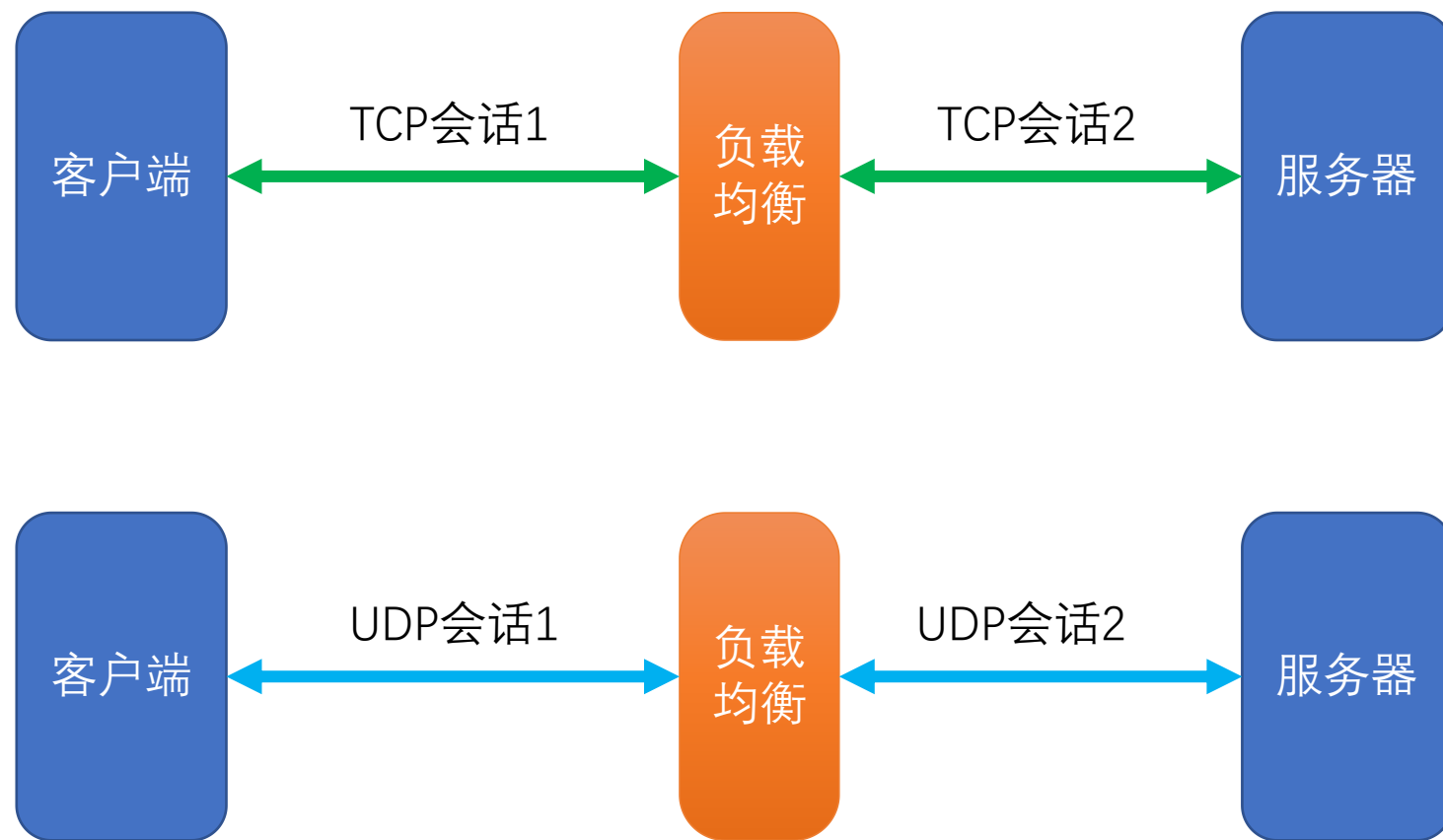
Initial[0]: CRYPTO[SH] ACK[0]
Handshake[0] CRYPTO[EE, FIN]
<- 1-RTT[0]: STREAM[1, "..."] ACK[0]

Initial[1]: ACK[0]
Handshake[0]: CRYPTO[FIN], ACK[0]
1-RTT[1]: STREAM[0, "..."] ACK[0] ->

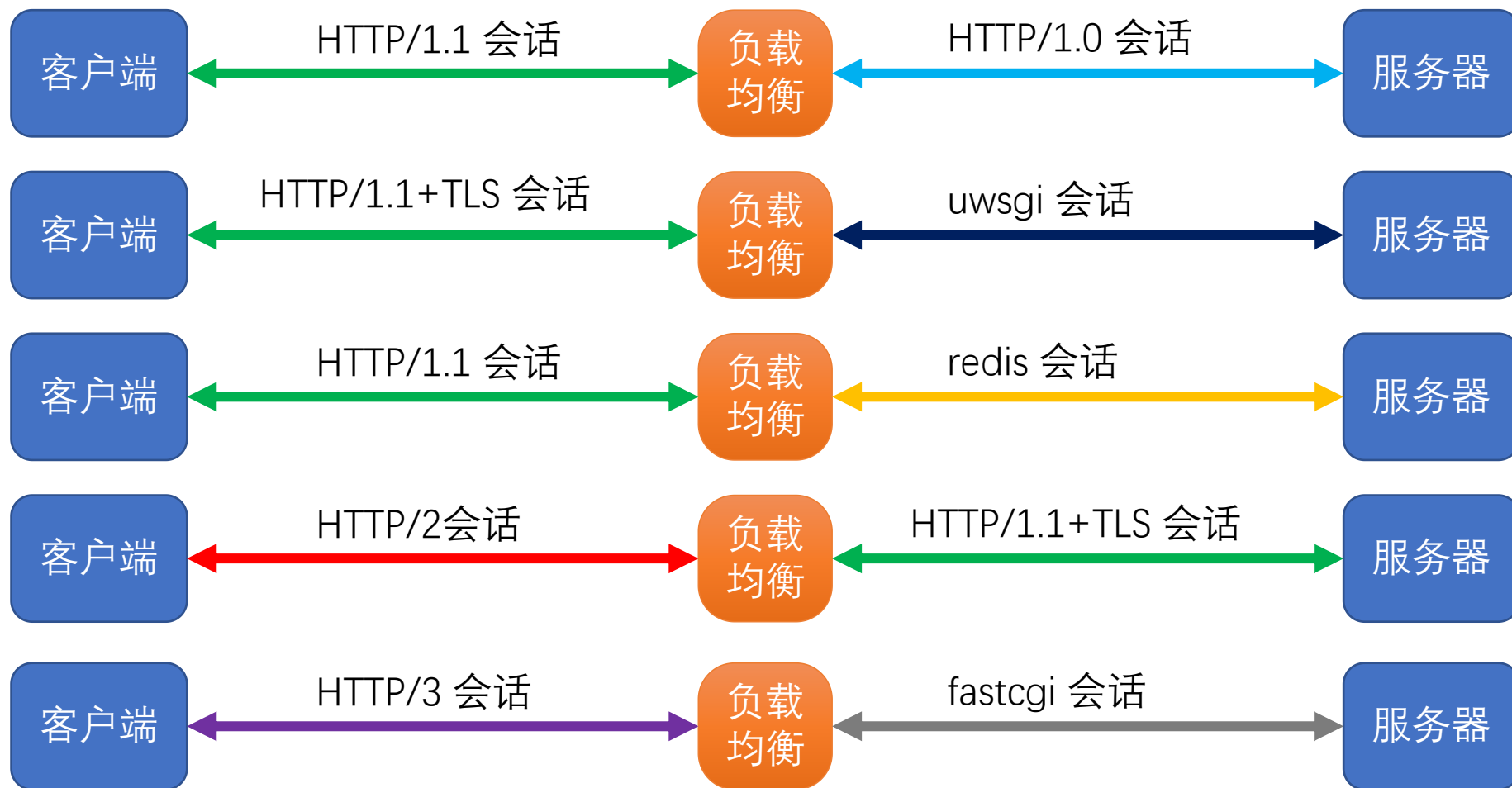
1-RTT[1]: STREAM[3, "..."], ACK[1]
<- Handshake[1]: ACK[0]

第 21 课 七层负载均衡做了些什么？

四层负载均衡



七层负载均衡协议转换举例



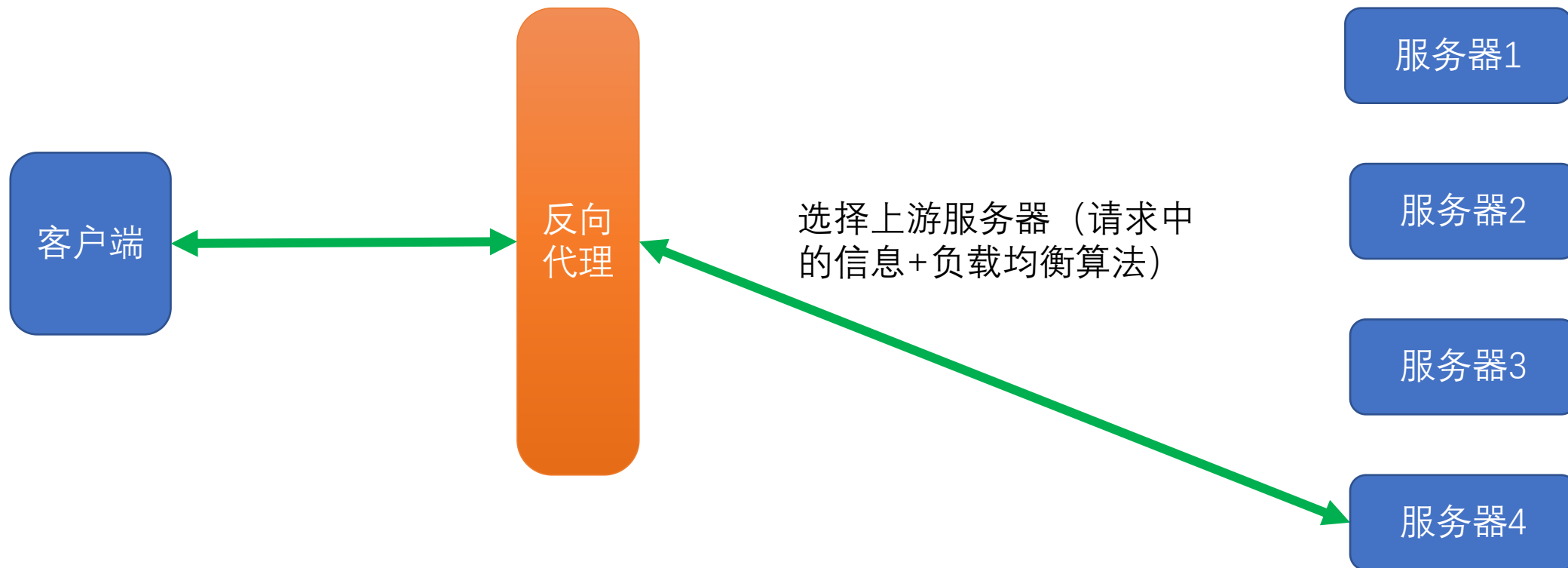
HTTP 协议转换

- request line 起始行
 - URL 重写 (包括 query 参数转换)
 - method 变换
 - http version 版本变换
- header 头部
 - header 名字、值作转换 (如 HTTP/2 索引表中查询头部, 转换为适配协议格式)
 - 负载均衡对 header 作修改
 - 隐藏某个 header (例如隐藏 X-Accel-Expires 等头部)
 - 新增 header (例如 CORS 允许跨域访问)
 - 修改 header 的 value 值 (例如修改 Server 头部的值)
- body 包体
 - 对内容使用通用算法 (如压缩算法) 转换
 - 按固定协议格式对内容进行转换

WAF 防火墙 (Web Application Firewall)

- request line 请求行
 - 检查 URL 及 query 参数是否合法 (如 SQL 注入)
 - method 方法是否合法 (例如阻止 TRACE 方法)
 - http version 版本是否合法 (例如不接收 HTTP/1.0 请求)
- header 头部
 - 检查 header 项是否符合应用场景要求
- body 包体
 - 对于FORM表单等通用格式做过滤

负载均衡算法



缓存功能

